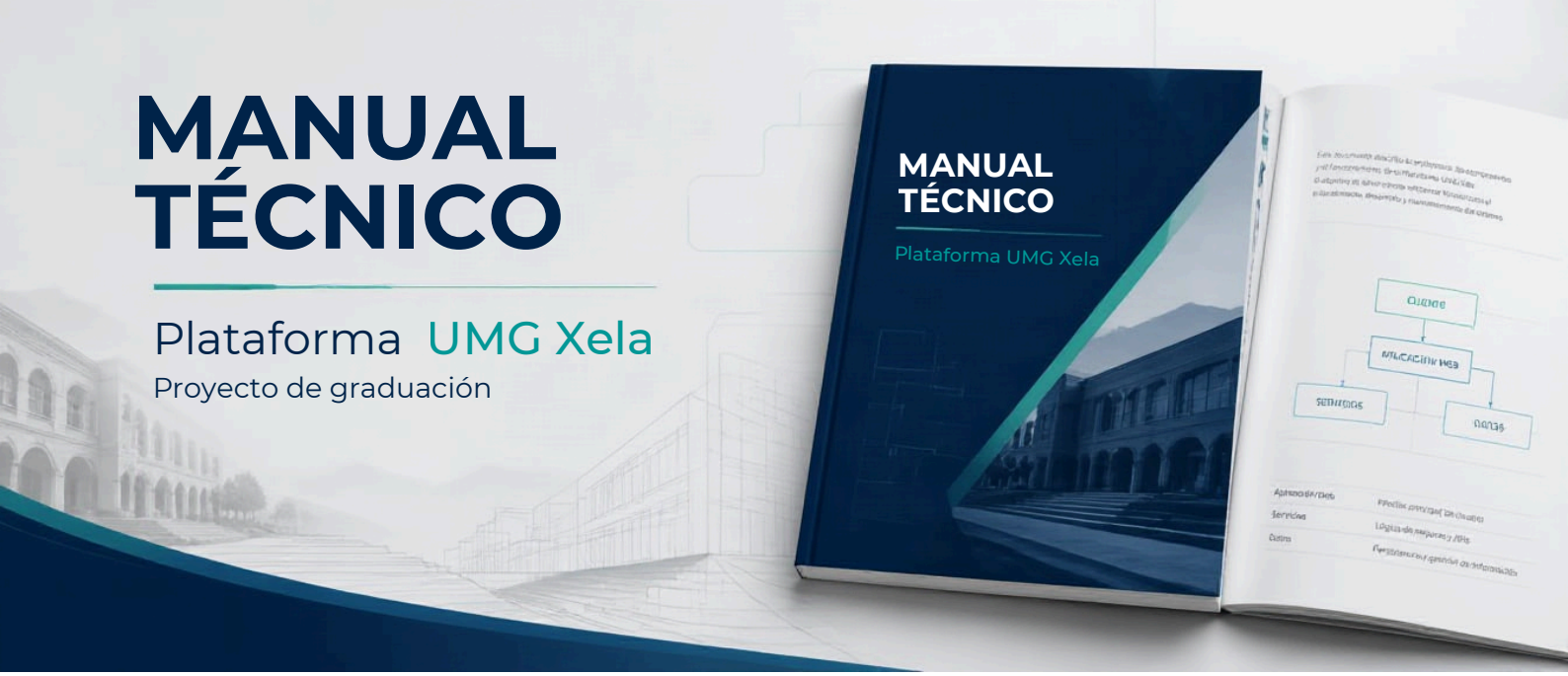


MANUAL TÉCNICO — Plataforma UMG Xela

Desarrollo de una plataforma informática preventiva y de orientación sobre suicidio y abuso sexual en el departamento de Quetzaltenango.

Tecnologías principales: *Laravel, PHP, MySQL, Firebase Authentication, Google Maps API, Git/GitHub* y *DigitalOcean.*

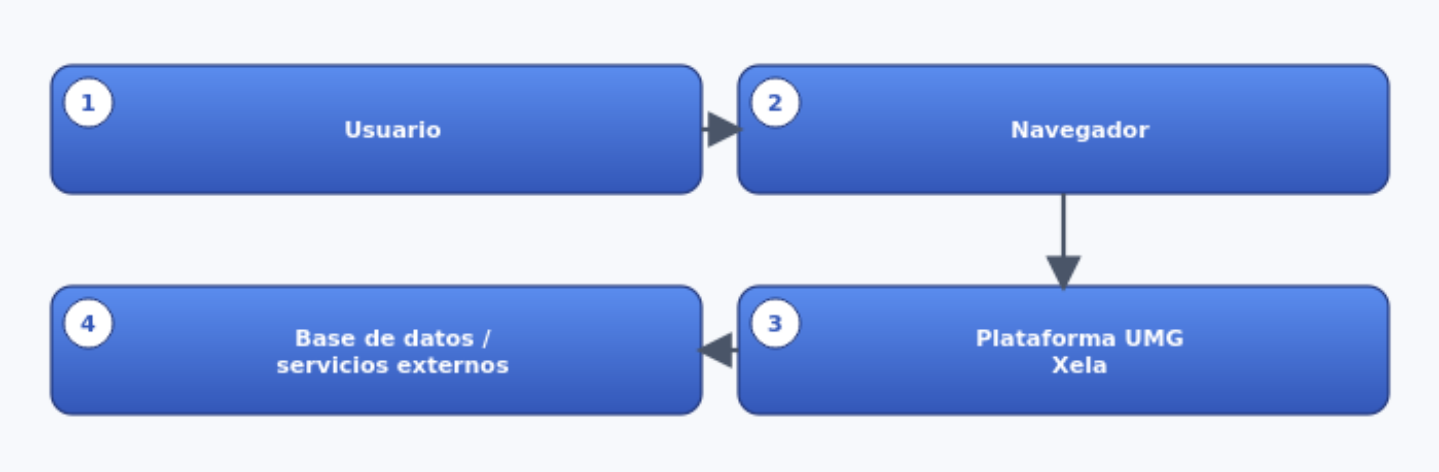
Tipo de proyecto: Proyecto de graduación.

INTRODUCCIÓN

Este manual describe los principales componentes tecnológicos, la estructura, la configuración y el funcionamiento de la Plataforma UMG Xela. Su propósito es presentar de forma ordenada los elementos que integran la aplicación y los servicios relacionados con su operación.

La documentación se organiza por secciones independientes para facilitar la consulta de la arquitectura, el código, la base de datos, la autenticación, los servicios externos, la configuración, el despliegue y el mantenimiento.

Esquema general



OBJETIVO DEL MANUAL

Objetivo: proporcionar una referencia técnica para comprender la estructura y funcionamiento de la plataforma, facilitando el mantenimiento, configuración, actualización y futura ampliación.

Alcance:

1. **Estructura general:** organización principal de la aplicación.
2. **Tecnologías:** herramientas y servicios utilizados por la plataforma.
3. **Organización del código:** distribución de componentes y archivos.
4. **Arquitectura:** interacción entre usuarios, aplicación, base de datos y servicios externos.
5. **Base de datos:** entidades, tablas y persistencia de información.
6. **Autenticación, usuarios y roles:** acceso y funciones generales de cada perfil.
7. **Servicios externos:** *Firebase Authentication*, *Google Maps API* y *DigitalOcean*.
8. **Configuración:** variables y parámetros del entorno.
9. **Control de versiones:** uso de *Git*/*GitHub*.
10. **Despliegue y mantenimiento:** operación, respaldo y continuidad técnica.

REQUISITOS DEL ENTORNO

La documentación del proyecto establece las siguientes tecnologías y recursos para el entorno de desarrollo y producción.

Elemento	Requisito o tecnología
Sistema operativo	*Windows*/*Linux*
Lenguaje	*PHP* 8.5
Framework	*Laravel*
Base de datos	*MySQL*
Administración BD	*MySQL Workbench*
Editor	*Visual Studio Code*
Dependencias *PHP*	*Composer*
Dependencias *frontend*	*NPM*
Autenticación	*Firebase Authentication*
Geolocalización	*Google Maps API*
Control de versiones	*Git*/*GitHub*
Producción	*DigitalOcean*

ESTRUCTURA DEL PROYECTO

La aplicación sigue la estructura de Laravel y utiliza el patrón Modelo-Vista-Controlador (MVC). Esta organización separa la lógica de la aplicación, las vistas, las rutas, la persistencia y los recursos necesarios para su ejecución.

plataforma/
├── *app*/
│ ├── *Http*/
│ ├── *Models*/
│ └── ...
├── *config*/
├── *database*/
│ ├── *migrations*/
│ └── *seeders*/
├── *public*/
├── *resources*/

```
│   └── views/
├── routes/
├── storage/
├── tests/
├── vendor/
├── .env
├── artisan
├── composer.json
└── package.json
```

***app/*:** contiene componentes principales de la aplicación, incluidos los controladores y modelos.

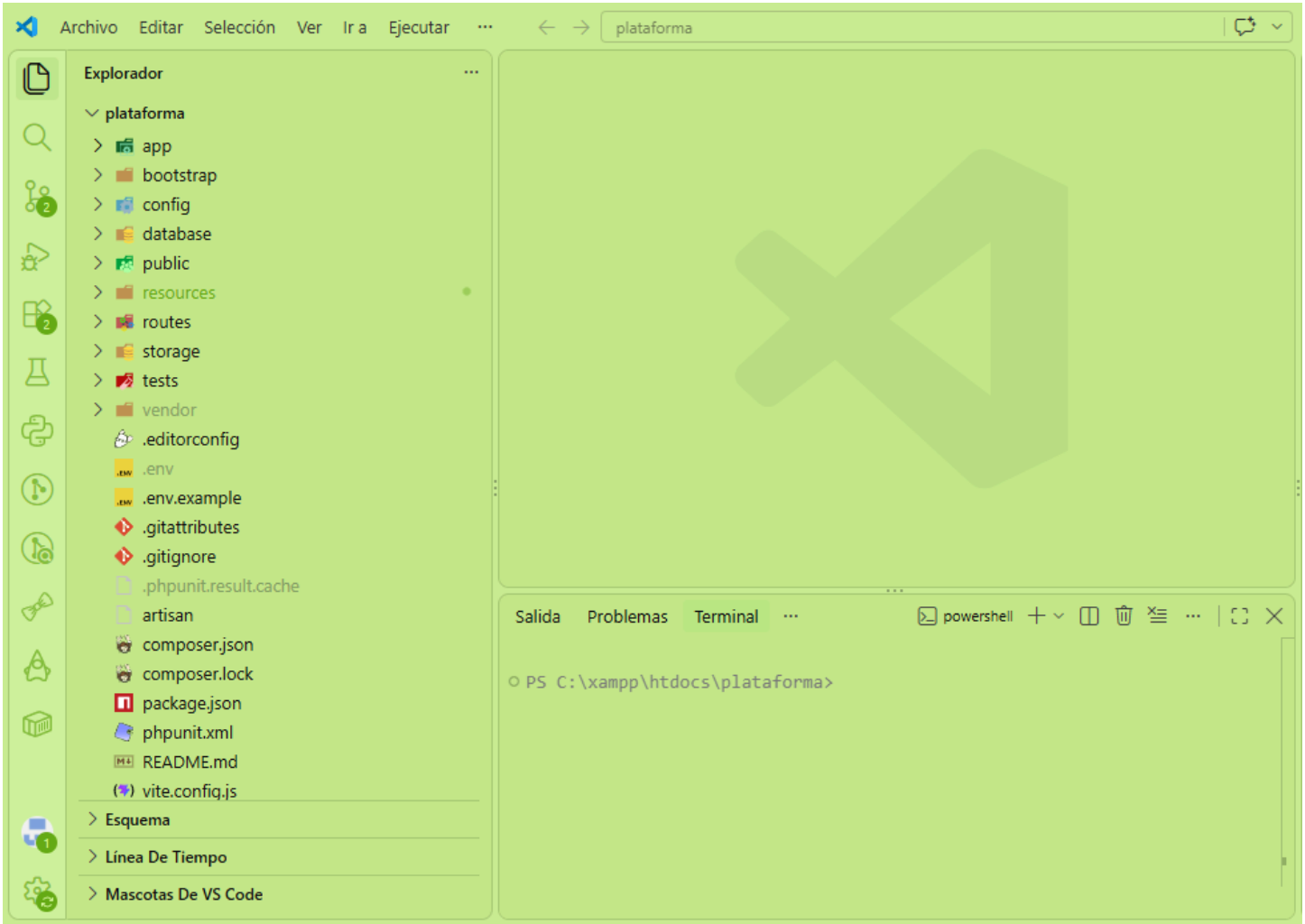
***resources/views/*:** contiene las vistas de la interfaz, construidas con *Blade, HTML, CSS* y *JavaScript*.

***routes/*:** contiene la definición de las rutas que reciben y dirigen las solicitudes.

***database/*:** contiene migraciones y *seeders* relacionados con la base de datos.

***public/*:** contiene los recursos públicos necesarios para servir la aplicación.

***storage/*:** contiene elementos de almacenamiento utilizados por la aplicación.



PATRÓN MVC

El patrón Modelo-Vista-Controlador organiza la aplicación en componentes con responsabilidades diferenciadas. *Laravel* utiliza esta separación para facilitar la comprensión, el mantenimiento y la evolución de la plataforma.

Modelos

Los modelos representan entidades y datos utilizados por la aplicación. Entre los modelos se encuentran *User*, *CaseFile*, Evaluacion, EvaluacionPregunta, EvaluacionRespuesta, *EvaluationAttempt* y *ModeratorNotification.*

Vistas

Las vistas se encuentran en *resources/views/* y utilizan *Blade* junto con *HTML, CSS* y *JavaScript* para construir la presentación de la información al usuario.

Controladores

Los controladores reciben solicitudes, coordinan el procesamiento y relacionan las vistas con los modelos. Los controladores principales son *AdminController, ModeratorController, EvaluacionController* y *UserController.*

FLUJO DE UNA SOLICITUD

Flujo general: Usuario → Navegador *web* → Ruta *Laravel* → *Middleware* → Controlador → Modelo / *Eloquent ORM* → *MySQL*

El *middleware* interviene antes de que la solicitud llegue al controlador. Su función es verificar condiciones de acceso y autorización definidas para la ruta correspondiente.

Ejemplo paso a paso

1. El usuario realiza una acción desde el navegador *web*.
2. El navegador envía la solicitud a la aplicación.
3. Laravel identifica la ruta asociada con la solicitud.
4. El middleware verifica las condiciones de acceso y autorización.
5. El controlador procesa la solicitud.
6. El modelo utiliza *Eloquent ORM* para interactuar con la base de datos *MySQL.*
7. El resultado se devuelve a la vista y se presenta en el navegador.

RUTAS Y CONTROLADORES

Las rutas definen los puntos de acceso de la aplicación y establecen qué controlador debe procesar cada solicitud. Los controladores organizan la lógica necesaria para responder y devolver la vista correspondiente.

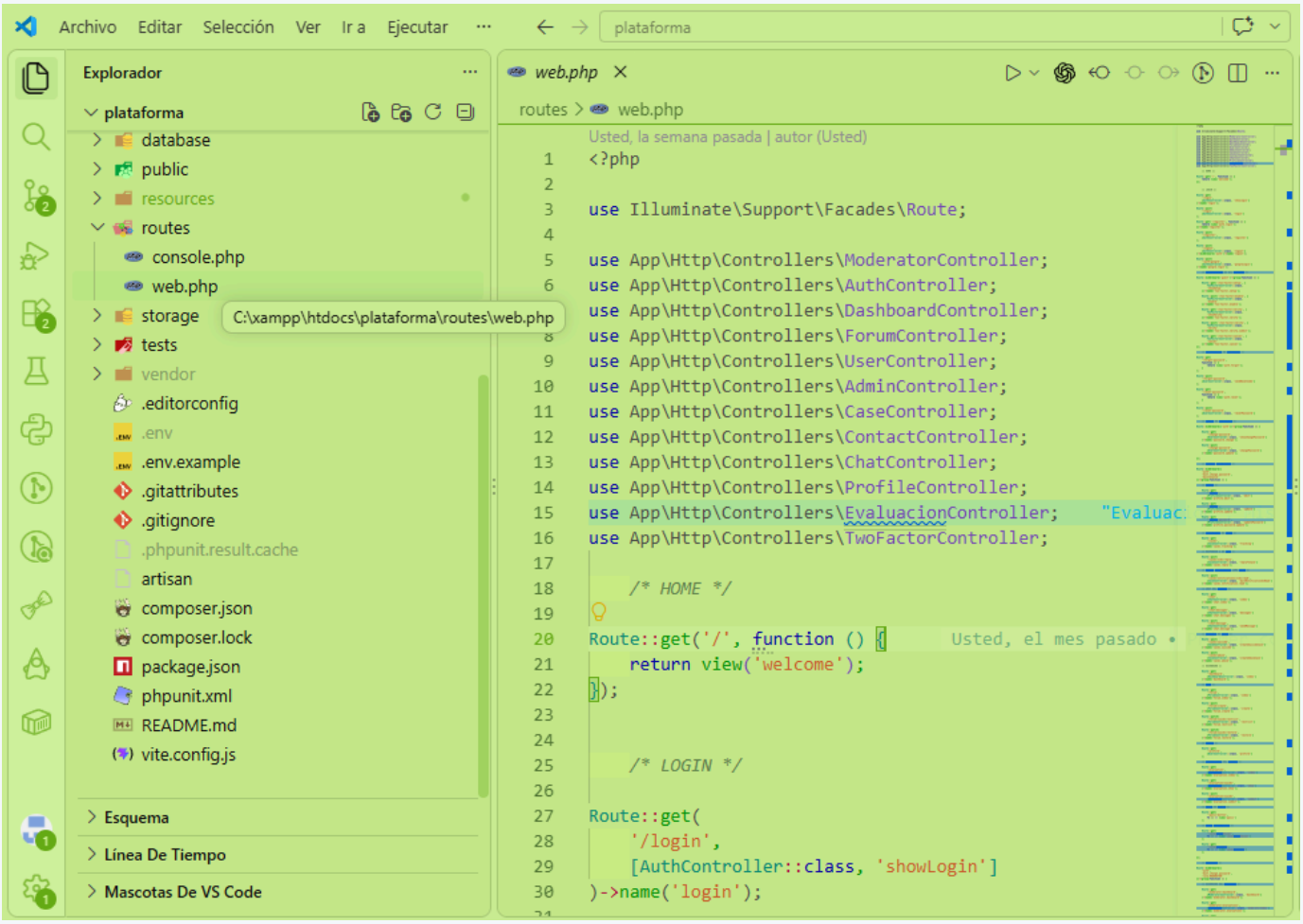
Ejemplo de ruta

```
Route::get(
'/moderator/evaluation-results/user/{userUuid}',
[EvaluacionController::class, 'moderatorUserResults']
)->name('moderator.evaluation-results.user');
```

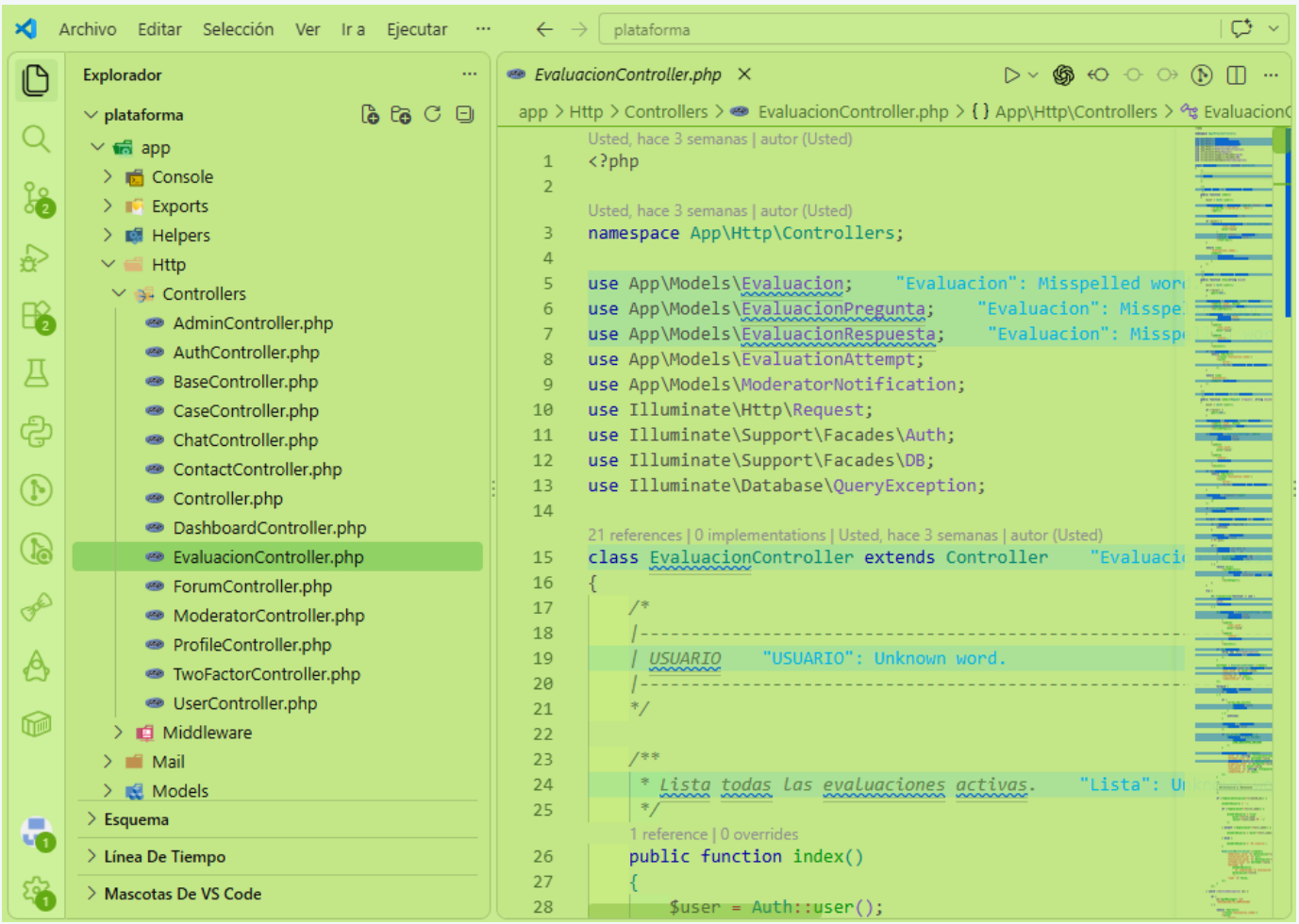
Esta ruta permite acceder a resultados de evaluaciones asociadas a un usuario.

Los controladores mencionados para la plataforma son M*oderatorController, EvaluacionController, AdminController y UserController.*

routes/web.php



app/Http/Controllers/EvaluacionController.php



AUTENTICACIÓN Y ROLES

Firebase Authentication se utiliza como servicio externo para el registro y el inicio de sesión. La autenticación verifica el acceso, mientras que la autorización determina las funciones disponibles para cada rol.

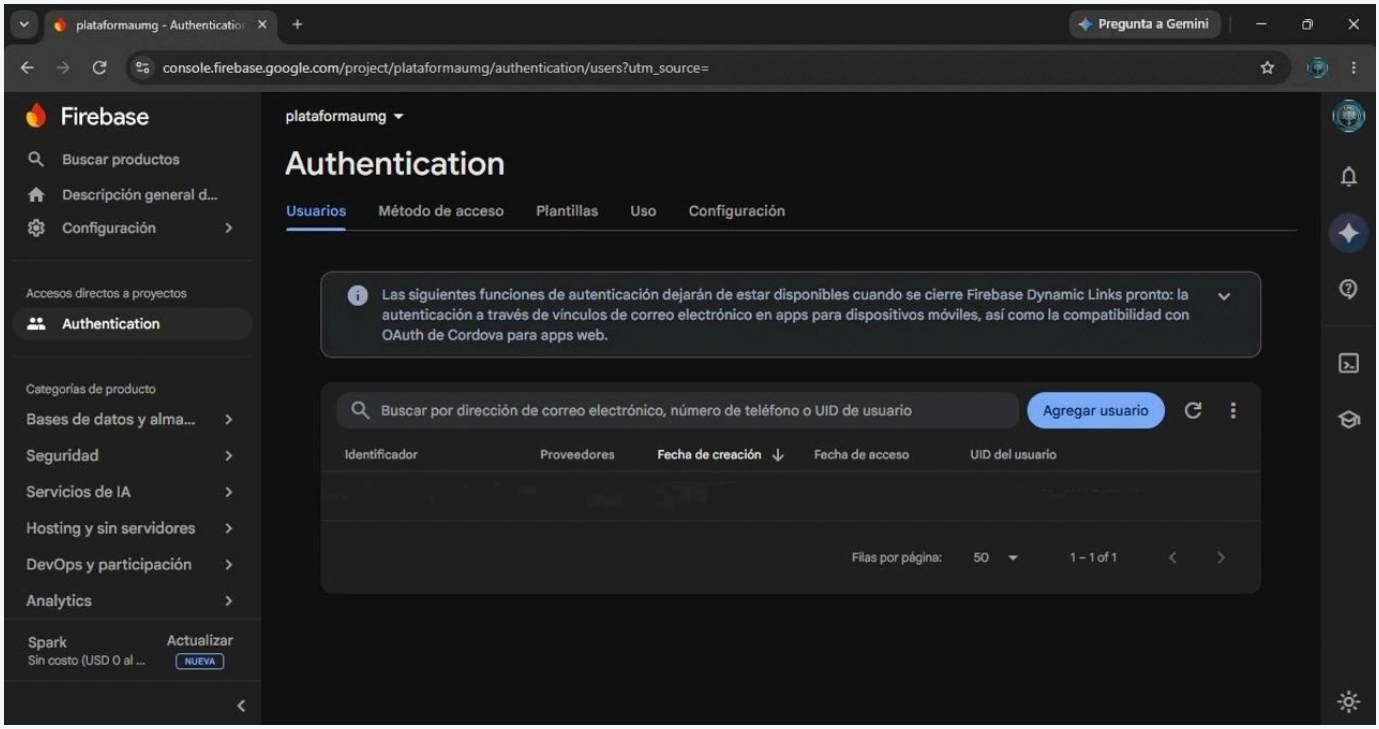
Rol	Funciones generales
USER	Utiliza funcionalidades para usuarios.
MODERATOR	Gestiona casos, evaluaciones, foro y conversaciones autorizadas.
ADMIN	Administra usuarios, moderadores, configuración, auditoría e informes.

La asignación de permisos se realiza en el servidor. Las restricciones de acceso no dependen únicamente de los elementos visibles en la interfaz, sino también de los mecanismos de *middleware* y autorización

implementados en *Laravel*.

Para reforzar la seguridad de las cuentas administrativas, se utiliza autenticación de dos factores mediante códigos temporales *TOTP* para los roles *ADMIN* y *MODERATOR*. El rol *USER* no requiere este segundo factor en la configuración actual. La sesión autenticada se establece después de completar las verificaciones exigidas para el rol correspondiente.

Firebase Console → Authentication



BASE DE DATOS

La plataforma utiliza *MySQL* para almacenar la información de usuarios, casos, conversaciones, evaluaciones, notificaciones, configuraciones y registros de auditoría..

Tablas de la plataforma

1. *users*
2. *case_files*
3. *case_comments*
4. *case_notifications*
5. *cases*
6. evaluaciones
7. *chat_conversations*
8. *chat_messages*
9. *crisis_reports*
10. mensajes_foro
11. *audit_logs*
12. *admin_notifications*

13. *moderator_notifications*
14. *user_changes*
15. *nickname_history*
16. *password_reset_codes*
17. *sessions*
18. *system_settings*

También existen tablas auxiliares para caché, migraciones y procesamiento de tareas, como *cache, cache_locks, migrations, jobs, job_batches* y *failed_jobs.*

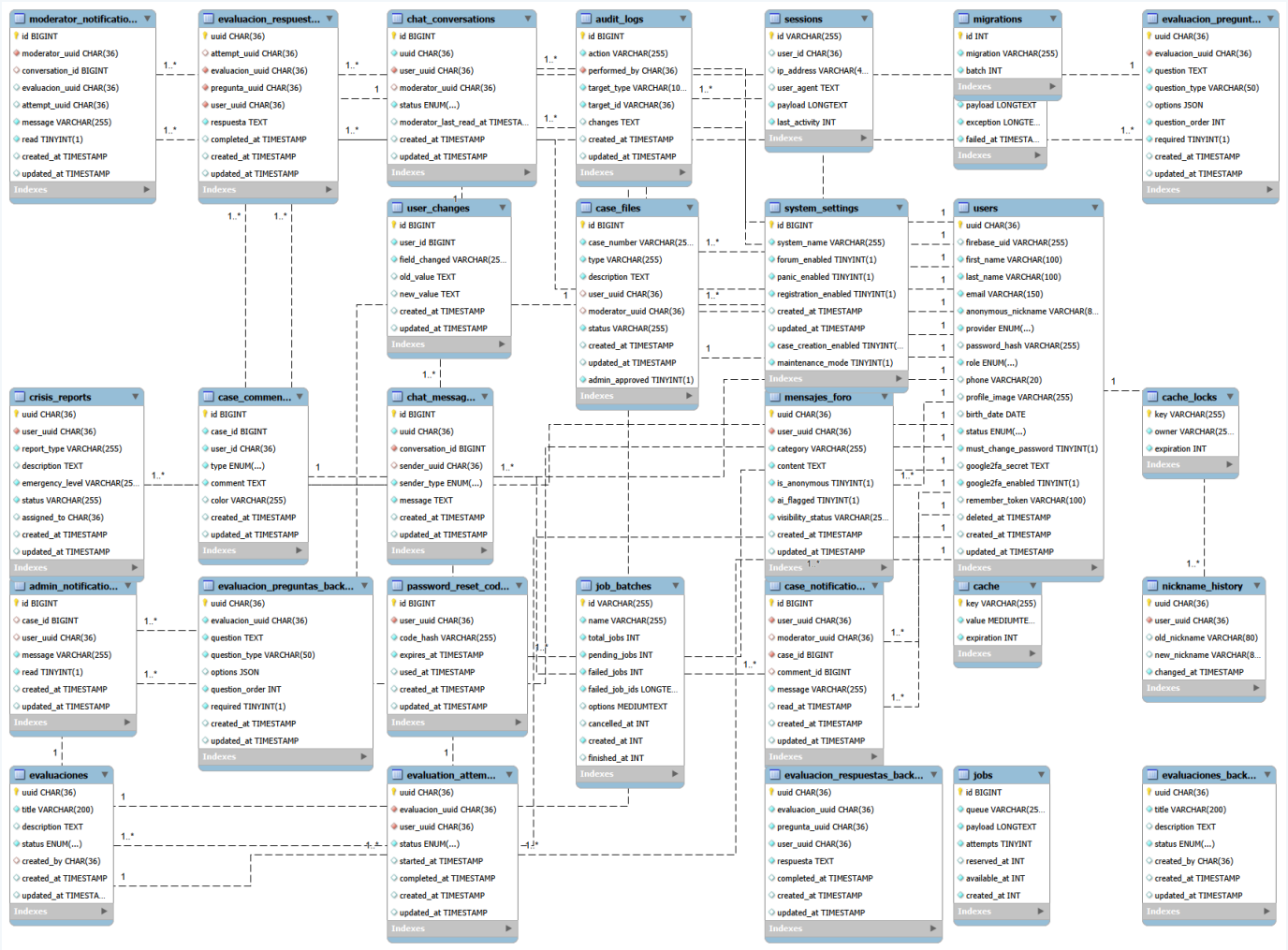
La estructura definitiva debe corresponder a las migraciones ejecutadas y a las tablas existentes en la base de datos de la versión entregada. Los nombres anteriores describen las tablas principales identificadas en el proyecto y no constituyen una sustitución de la verificación del esquema real.

MODELO DE DATOS

El modelo de datos reúne las entidades principales que permiten gestionar cuentas, casos, evaluaciones, comunicación y auditoría dentro de la plataforma.

Entidad o conjunto	**Descripción**
users	Contiene información de las cuentas y del acceso.
case_files	Representa los casos gestionados.
evaluaciones, evaluacion_preguntas, *evaluation_attempts* y evaluacion_respuestas	Gestionan evaluaciones, preguntas, intentos y respuestas.
chat_conversations y *chat_messages*	Registran conversaciones y mensajes de comunicación.
audit_logs	Registra información relacionada con la auditoría.

Diagrama de base datos



EVALUACIONES

El módulo de evaluaciones permite organizar evaluaciones, preguntas, respuestas e intentos. El proceso conecta la configuración realizada por el moderador con la participación del usuario y la consulta posterior de resultados.

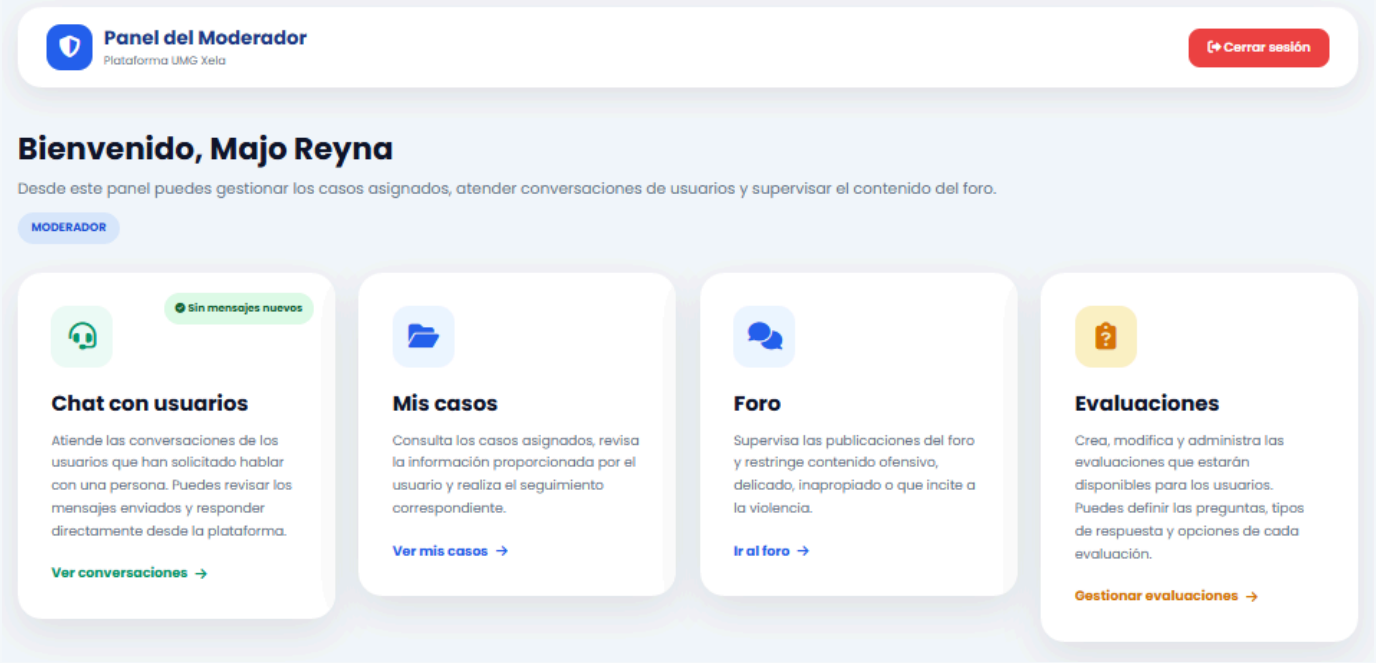
Proceso de evaluación

1. Crear evaluación
2. Agregar preguntas
3. Publicar evaluación
4. Usuario responde
5. Registrar intento
6. Guardar respuestas
7. Notificar al moderador
8. Consultar resultados

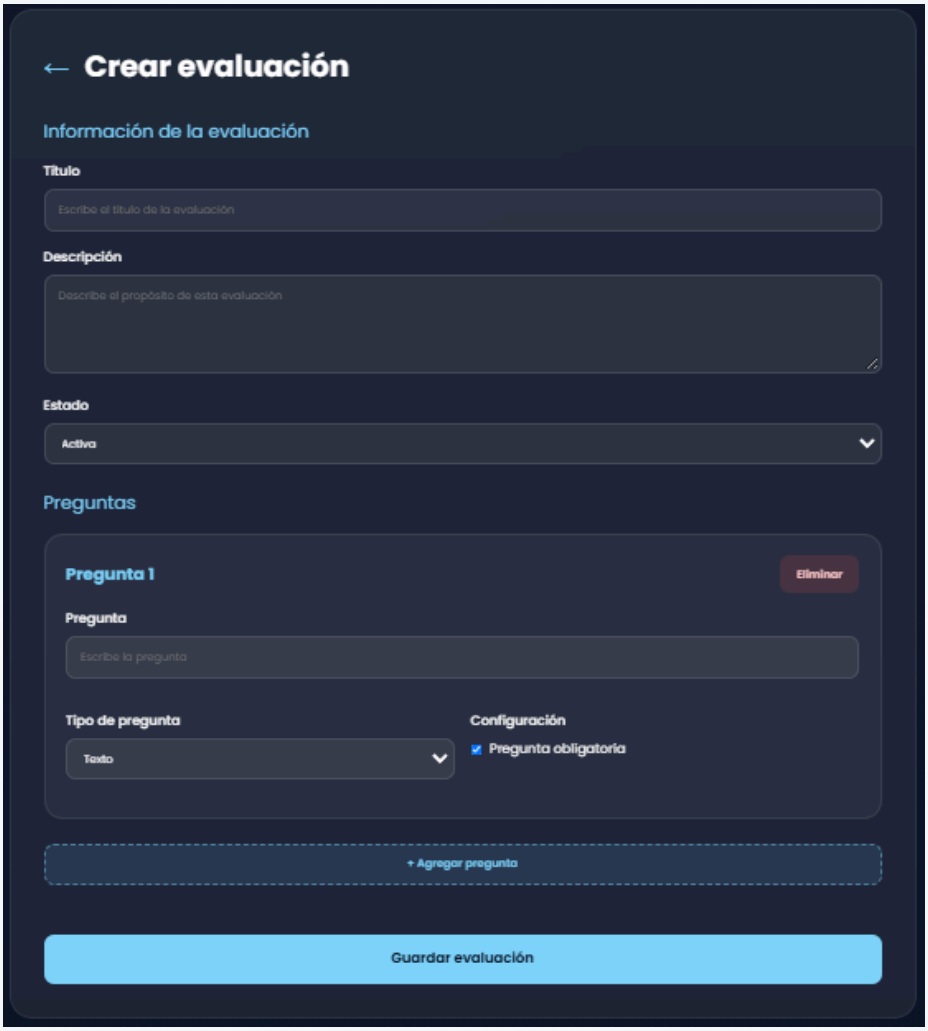
Relación textual: Evaluación → Preguntas e Intentos → Respuestas

El sistema verifica las preguntas obligatorias antes de completar el proceso de respuesta.

Panel del moderador



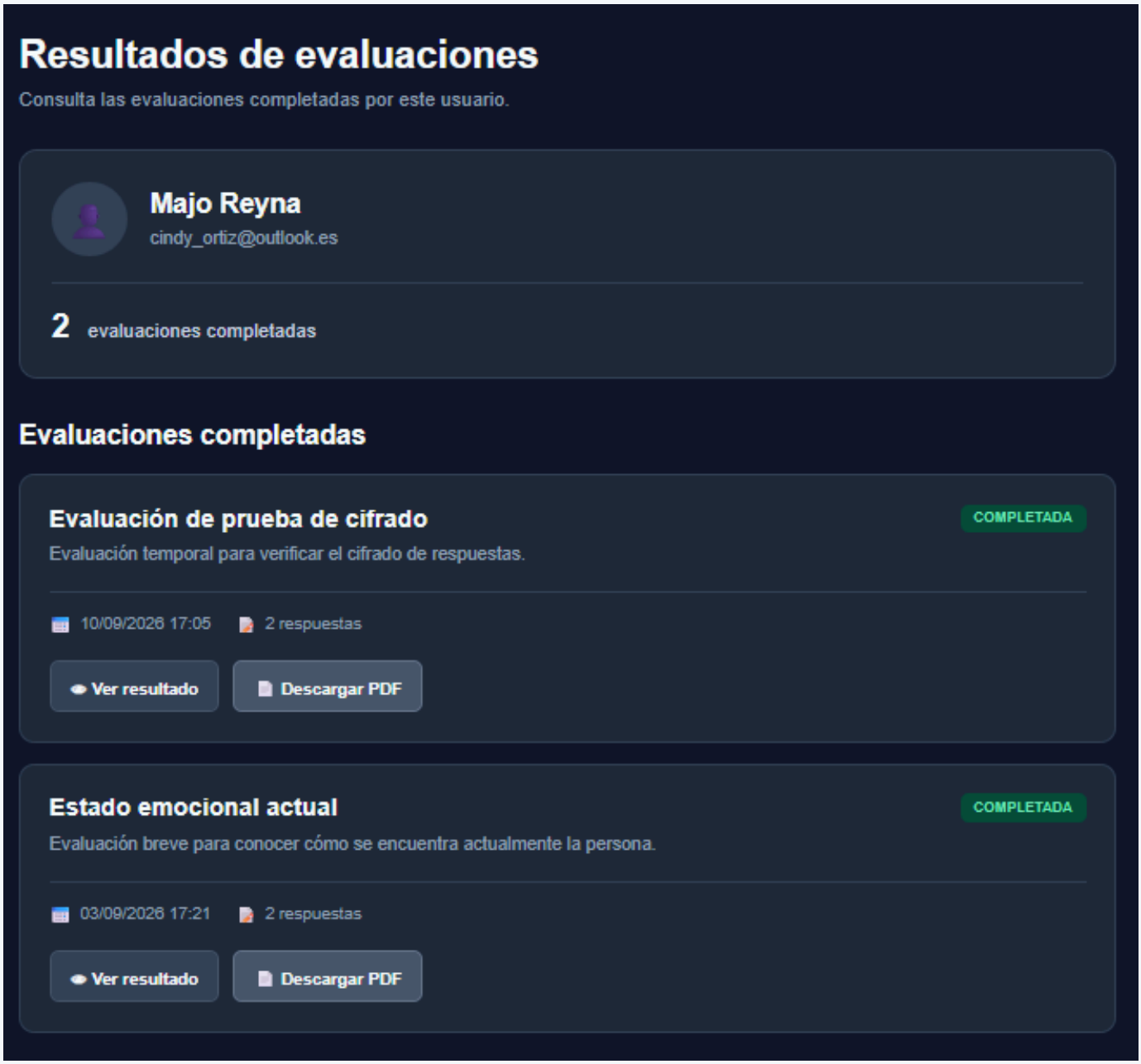
Creación de evaluación



Vista del usuario



Resultados



NOTIFICACIONES DE EVALUACIONES

Las notificaciones de evaluaciones informan al moderador cuando existe un resultado disponible para consulta.

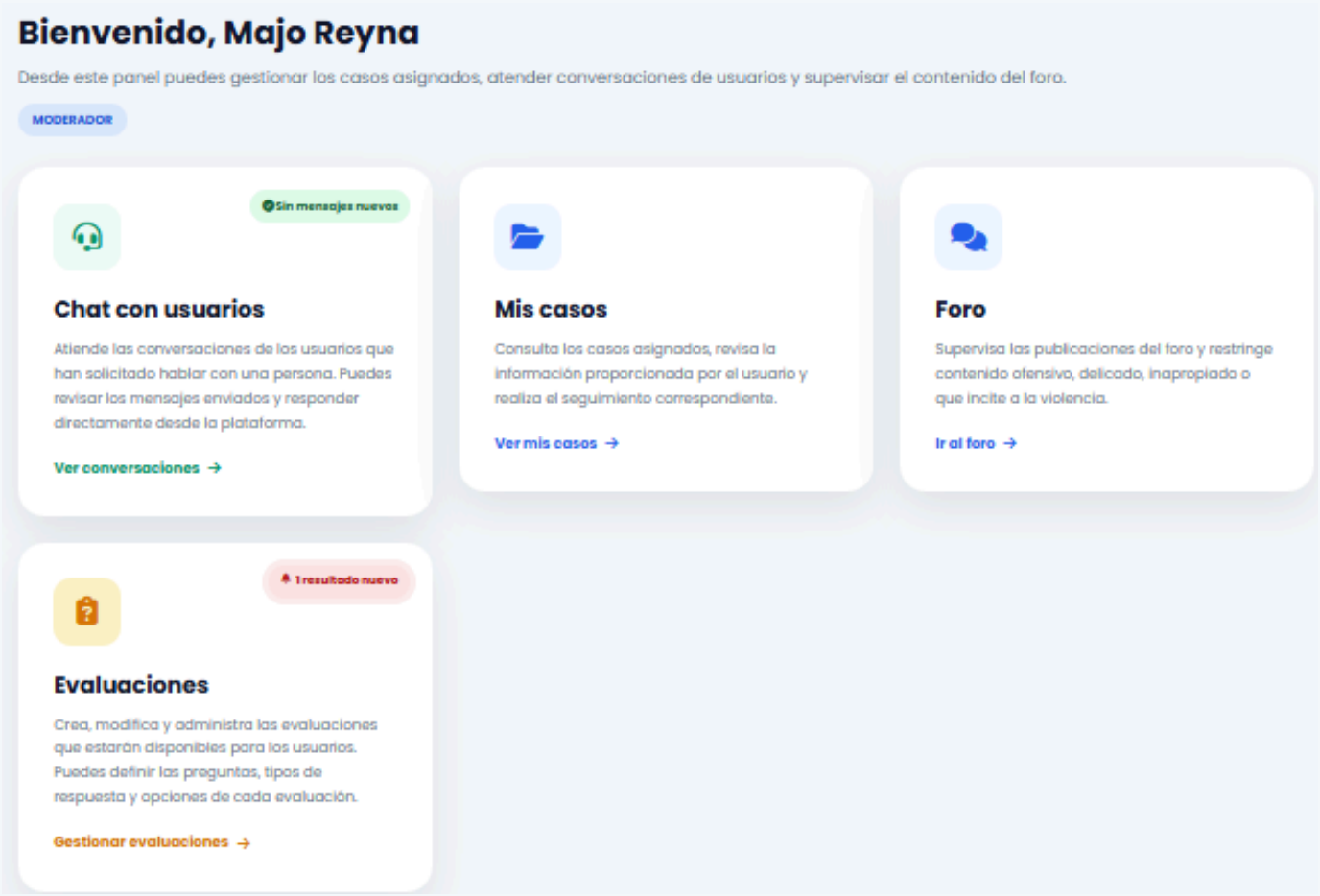
Flujo de notificación

1. El usuario completa la evaluación.
2. Se crea *EvaluationAttempt*.
3. Se registran las respuestas.
4. Se crea *ModeratorNotification.*
5. El moderador visualiza la notificación.
6. El moderador consulta los resultados.

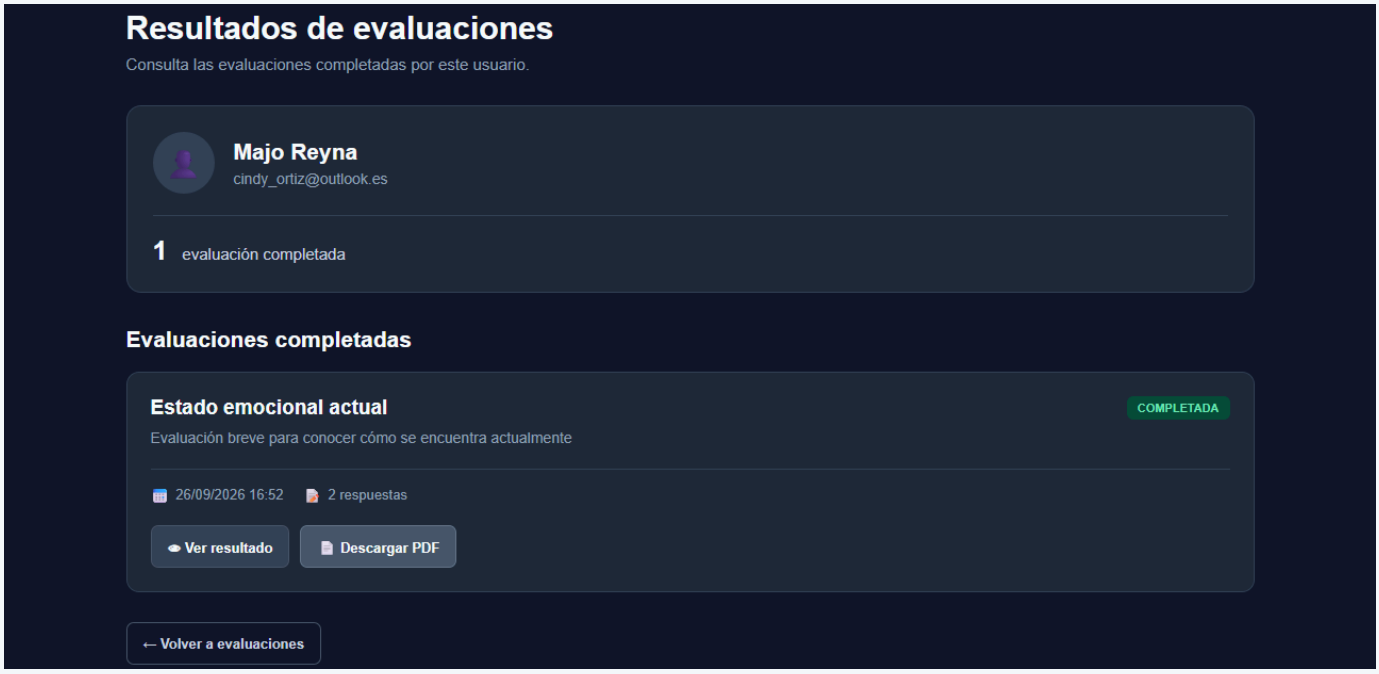
7. La notificación se marca como leída.

El estado de lectura se controla mediante el campo *read*.

“Evaluaciones” con “1 resultado nuevo”



Vista “Evaluaciones → Ver resultados”



FORO Y CHAT

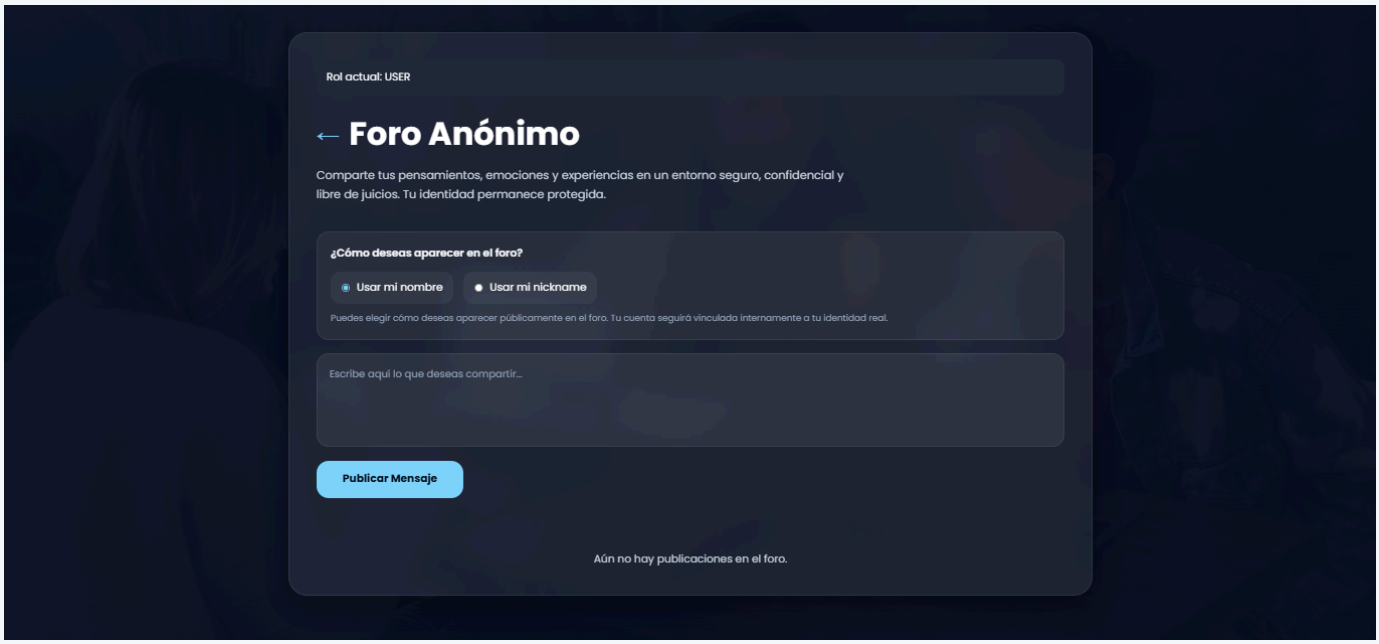
El foro permite la publicación de contenido por parte del usuario. El contenido pasa por validación, registro, moderación y visualización dentro de la plataforma.

Flujo del foro: Usuario → Publicación → Validación → Registro → Moderación → Visualización

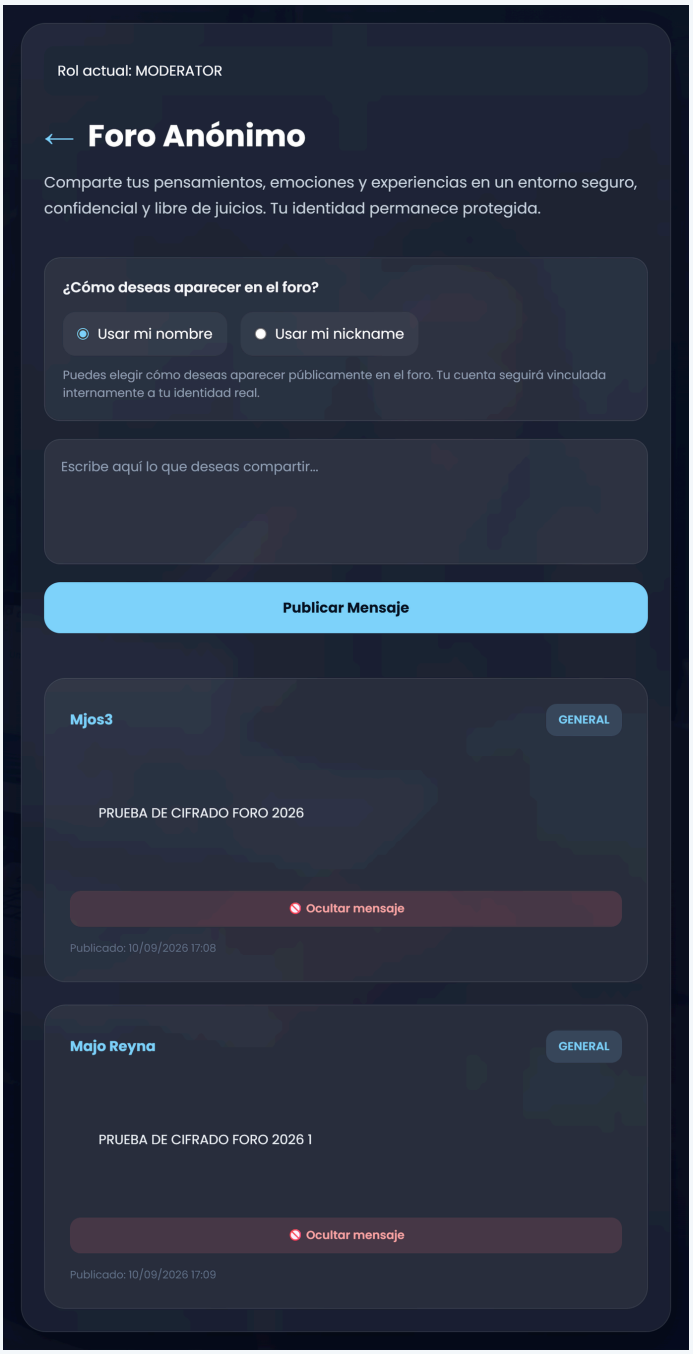
La publicación se registra y queda sujeta a las acciones de moderación correspondientes antes de su visualización según el funcionamiento autorizado de la plataforma.

El chat utiliza las tablas *chat_conversations* y *chat_messages* para organizar conversaciones y mensajes. El módulo contempla conversaciones asignadas y respuestas del moderador.

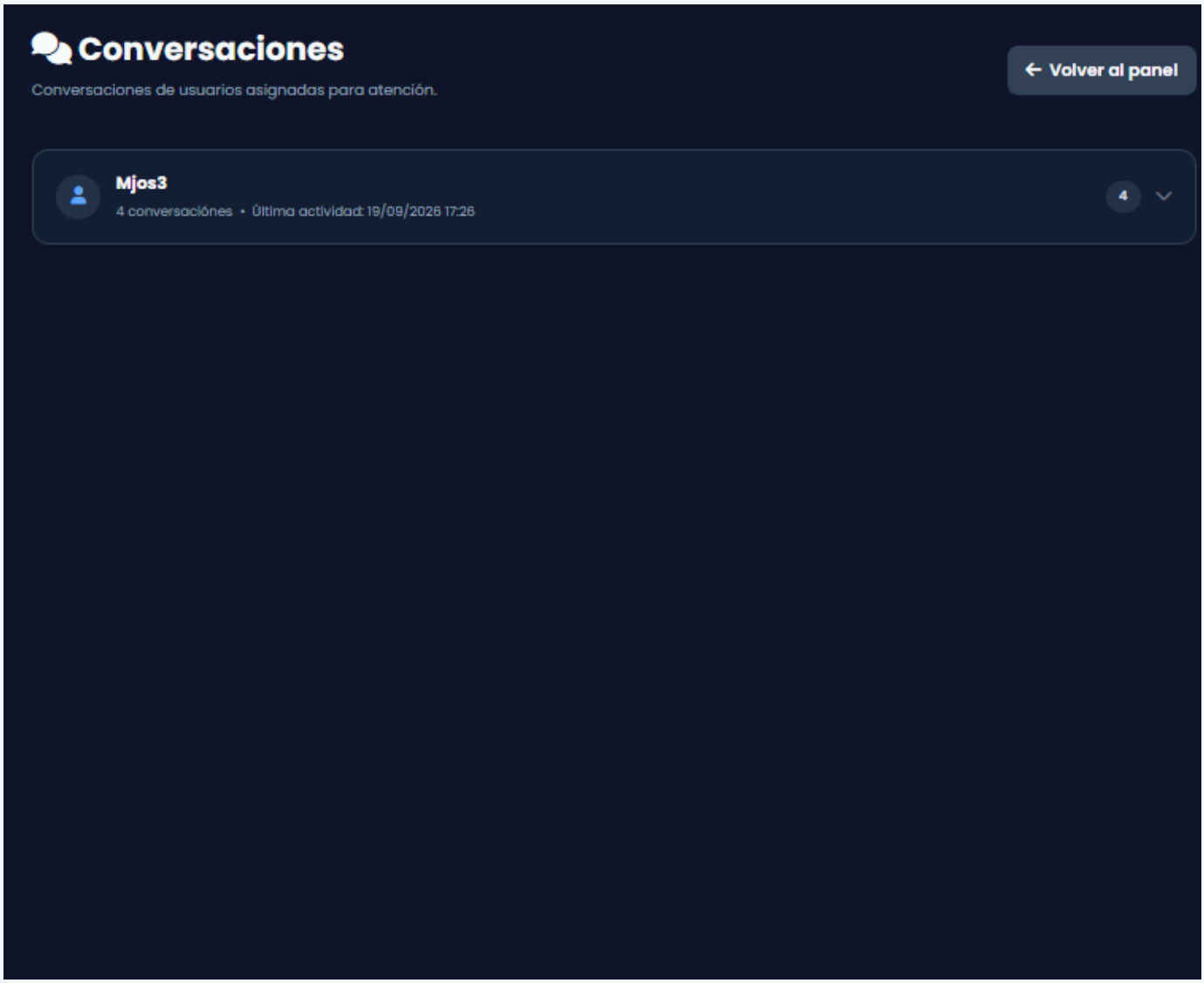
Vista del foro



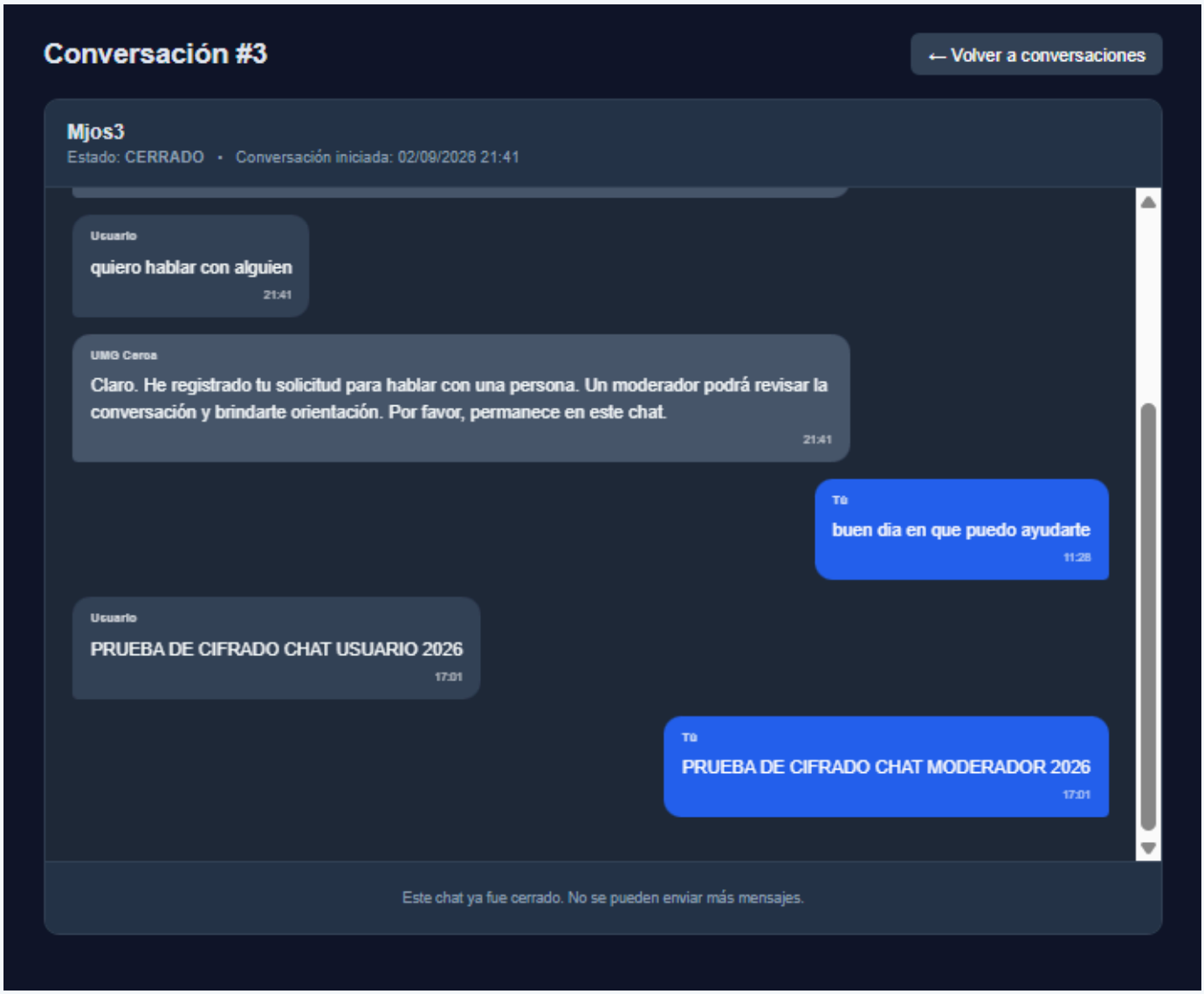
Panel de moderación



Panel de chat



Una conversación



CASOS Y REPORTES DE CRISIS

La plataforma permite gestionar y dar seguimiento a casos y reportes de crisis. La información queda almacenada para los usuarios autorizados.

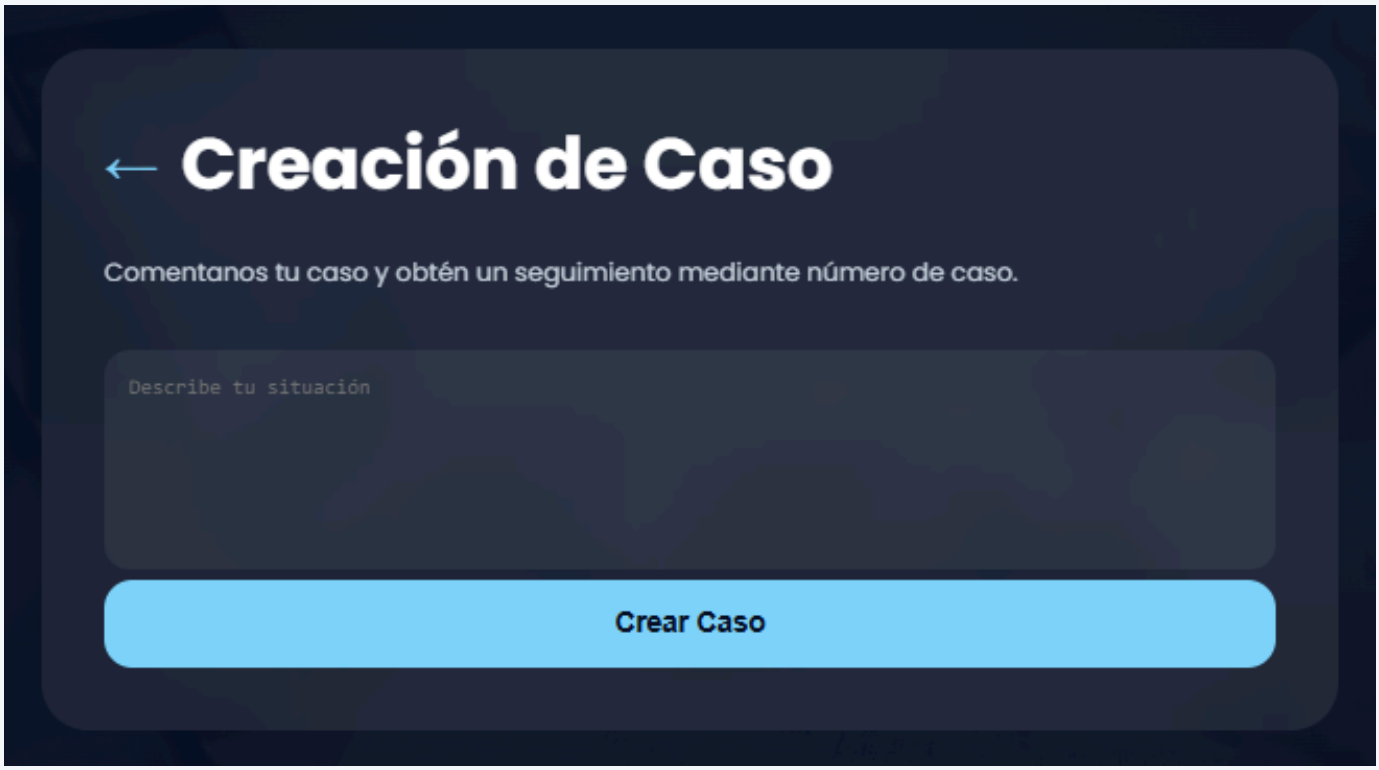
Estados de los casos:

1. Abierto
2. En seguimiento
3. Cerrado

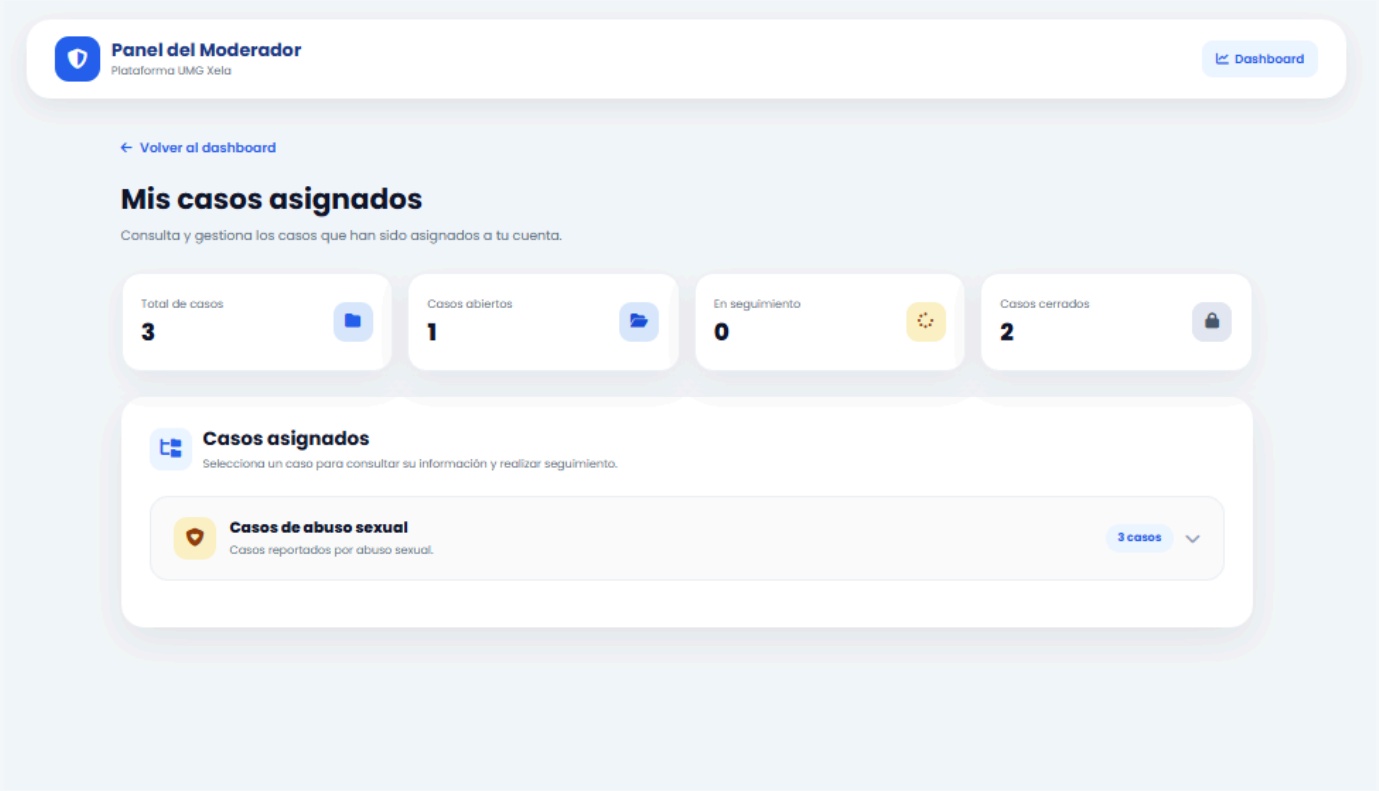
Flujo de gestión: Usuario → Reporte → Registro → Clasificación → Seguimiento

El seguimiento se realiza de acuerdo con el estado asignado al caso y con las funciones disponibles para los usuarios autorizados.

Creación de caso/reporte por el usuario



Listado de casos



Detalle de caso

Panel del Moderador
Plataforma UMG Xela

← Volver a mis casos

Detalle del caso

Información y seguimiento del caso asignado.

Información del caso

Datos registrados por el usuario.

Número de caso: VS-2026-1219	Tipo de caso: Abuso sexual
Usuario: Mjos3	Fecha de creación: 18/09/2026 13:42
Estado actual: Abierto	Moderador asignado: Majo Reyna
Última actualización: 18/09/2026 13:42	

Situación reportada por el usuario

Información proporcionada al registrar el caso.

Situación reportada

Caso de demostración para verificar el funcionamiento del registro y seguimiento del caso

Actualizar caso

Selecciona el estado y registra un comentario de seguimiento para el usuario.

Estado del caso

Abierto

Comentario de seguimiento

Escribe aquí la respuesta, orientación o seguimiento para el usuario...

Este comentario quedará registrado en el historial del caso y podrá ser consultado por el usuario como parte del seguimiento.

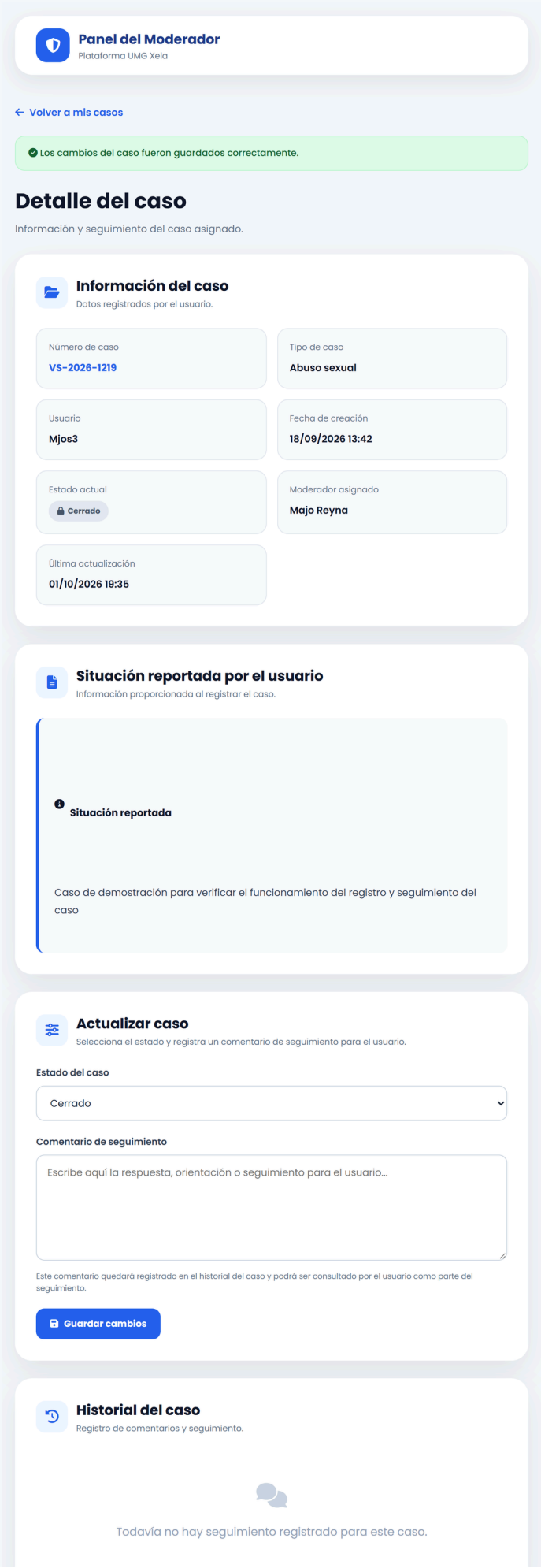
Guardar cambios

Historial del caso

Registro de comentarios y seguimiento.

Todavía no hay seguimiento registrado para este caso.

Cambio de estado por el moderador



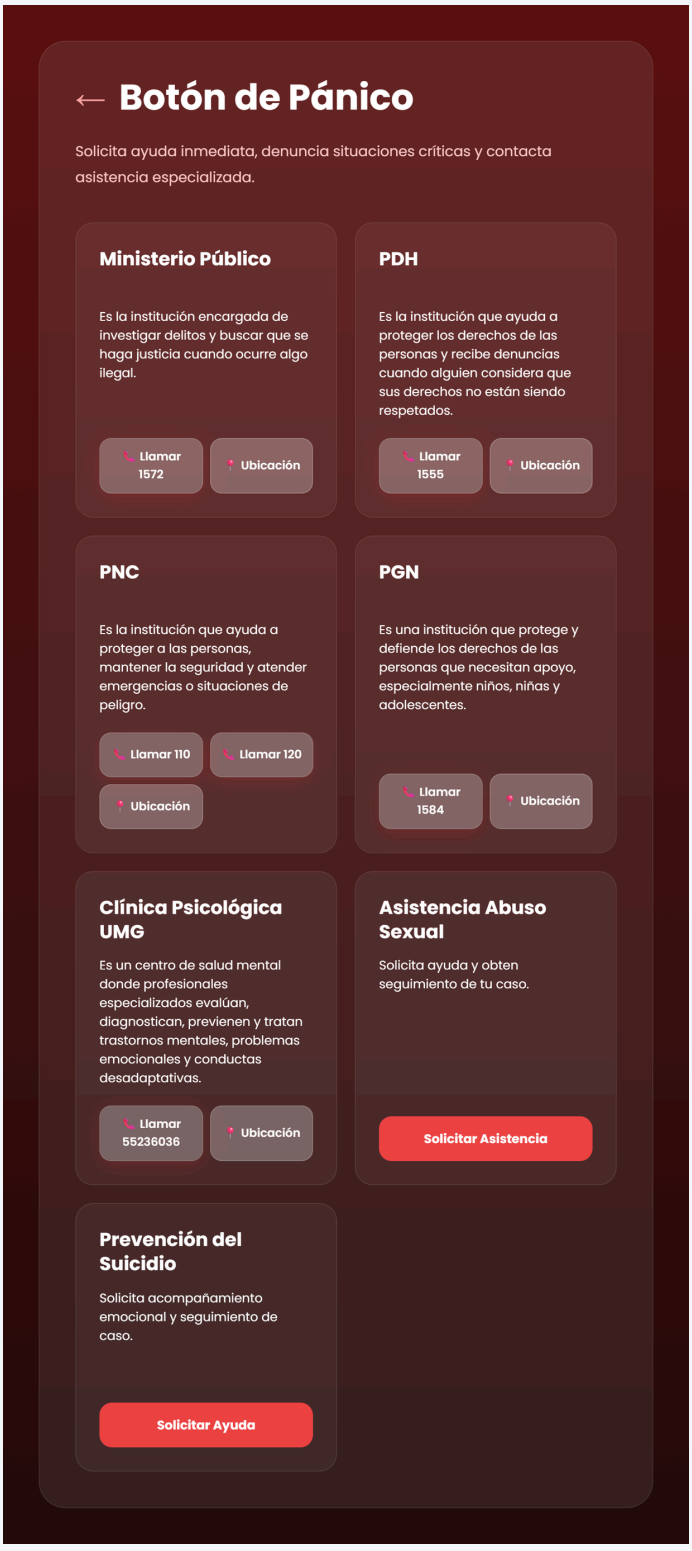
RECURSOS GEOLOCALIZADOS

La Plataforma UMG Xela integra *Google Maps API* para mostrar instituciones y organizaciones de apoyo en Quetzaltenango.

Flujo de consulta: Usuario → Consulta recursos → Obtención de ubicación → *Google Maps API* → Visualización de recursos

La integración permite presentar los recursos geolocalizados dentro de la plataforma de acuerdo con la consulta realizada por el usuario.

Mapa con recursos geolocalizados



SERVICIOS EXTERNOS

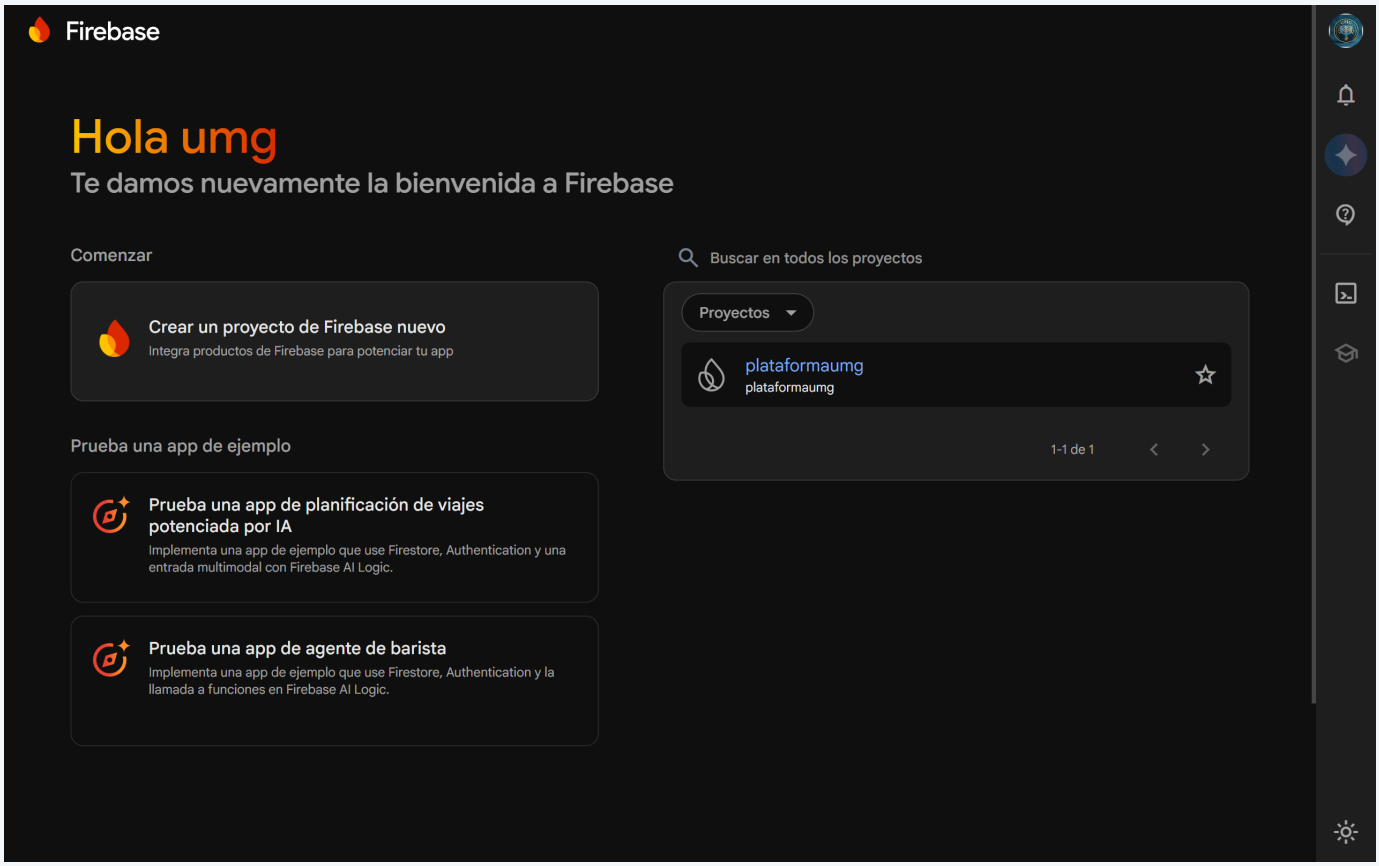
La plataforma utiliza servicios externos para funciones específicas de autenticación, geolocalización e infraestructura de producción.

Servicio	Función
Firebase Authentication	Registro e inicio de sesión.
Google Maps API	Visualización de recursos geolocalizados.
DigitalOcean	Infraestructura prevista para producción.

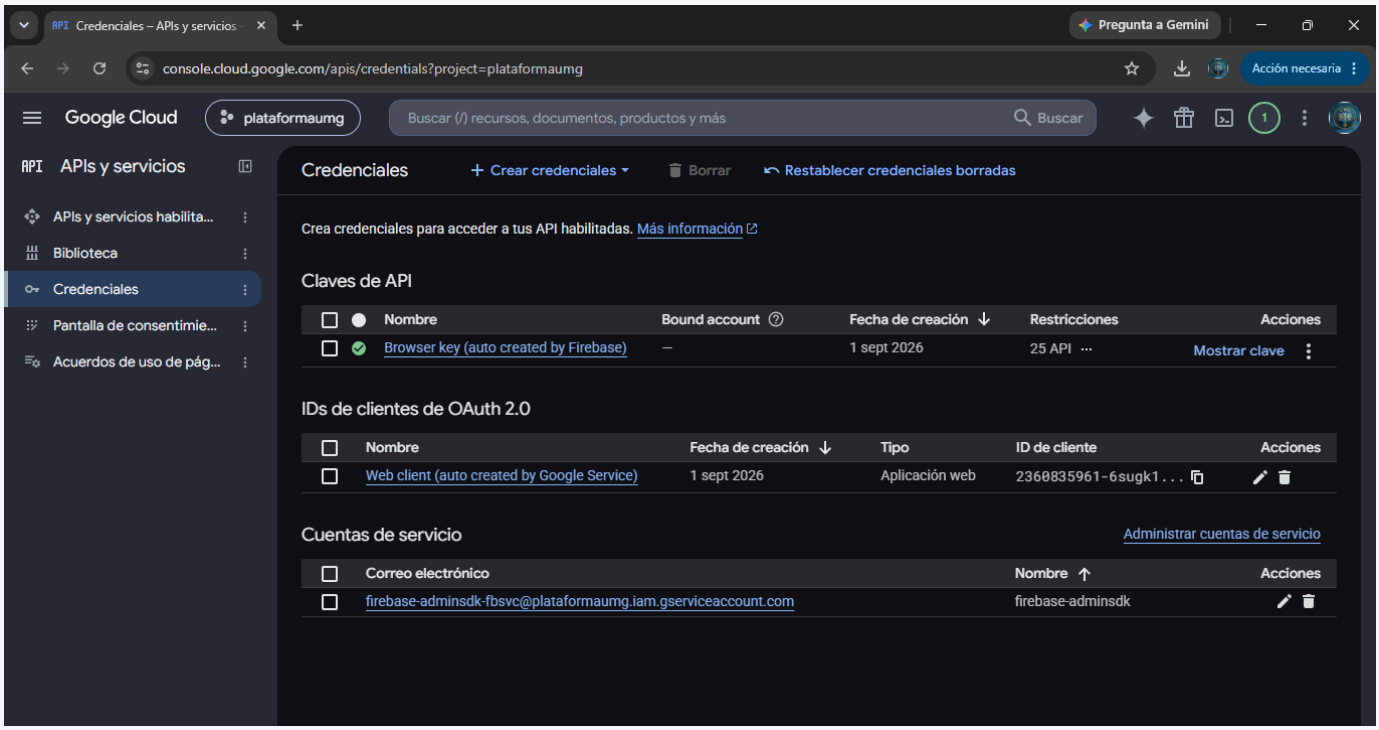
La producción contempla *DigitalOcean* sobre infraestructura Linux para ejecutar *Laravel.*

PLATAFORMA UMG XELA
- ├── *Firebase Auth*
- ├── *Google Maps API*
- ├── Aplicación
- ├── *DigitalOcean*
- └── *MySQL*

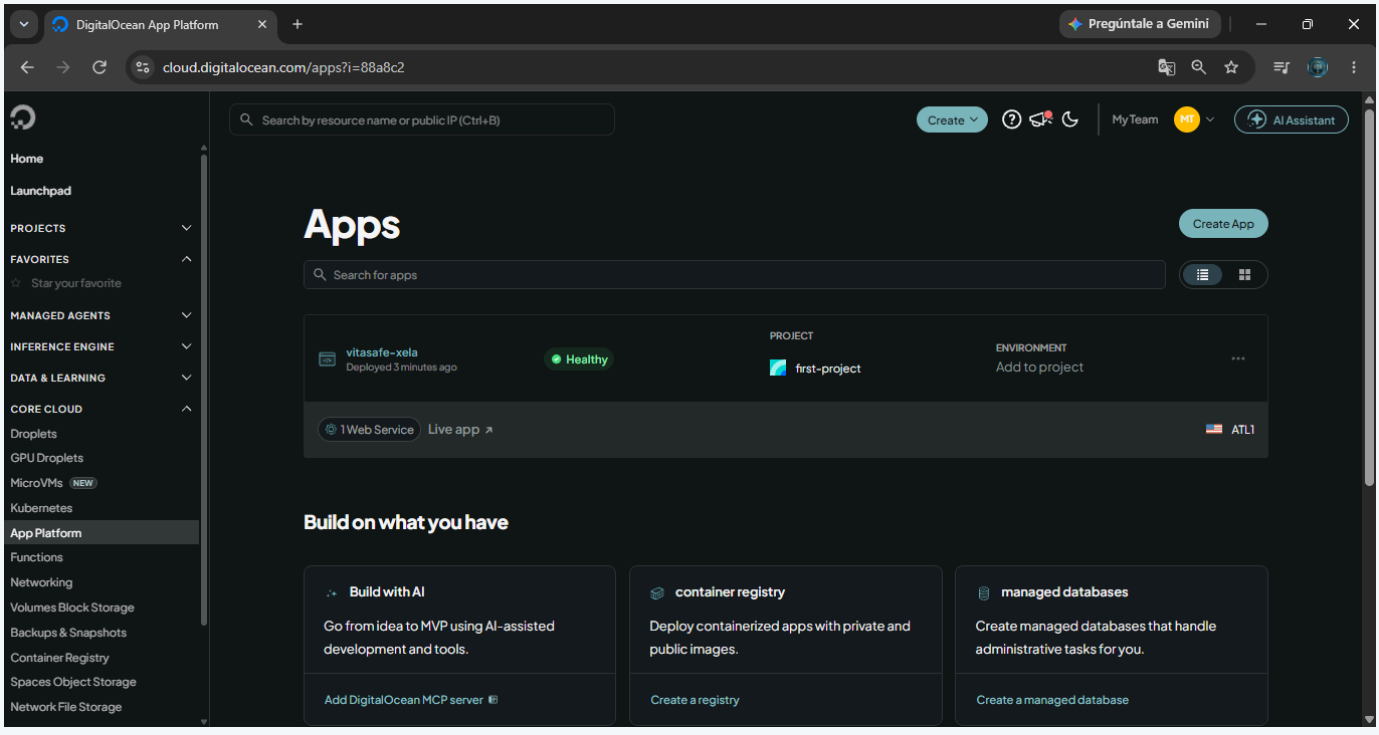
Firebase Console



Google Cloud/Maps sin claves



Panel de DigitalOcean



CONFIGURACIÓN DEL ENTORNO

El archivo *.env* contiene variables de configuración del entorno de la aplicación. Estas variables permiten definir parámetros de la aplicación, la conexión con la base de datos y los servicios externos.

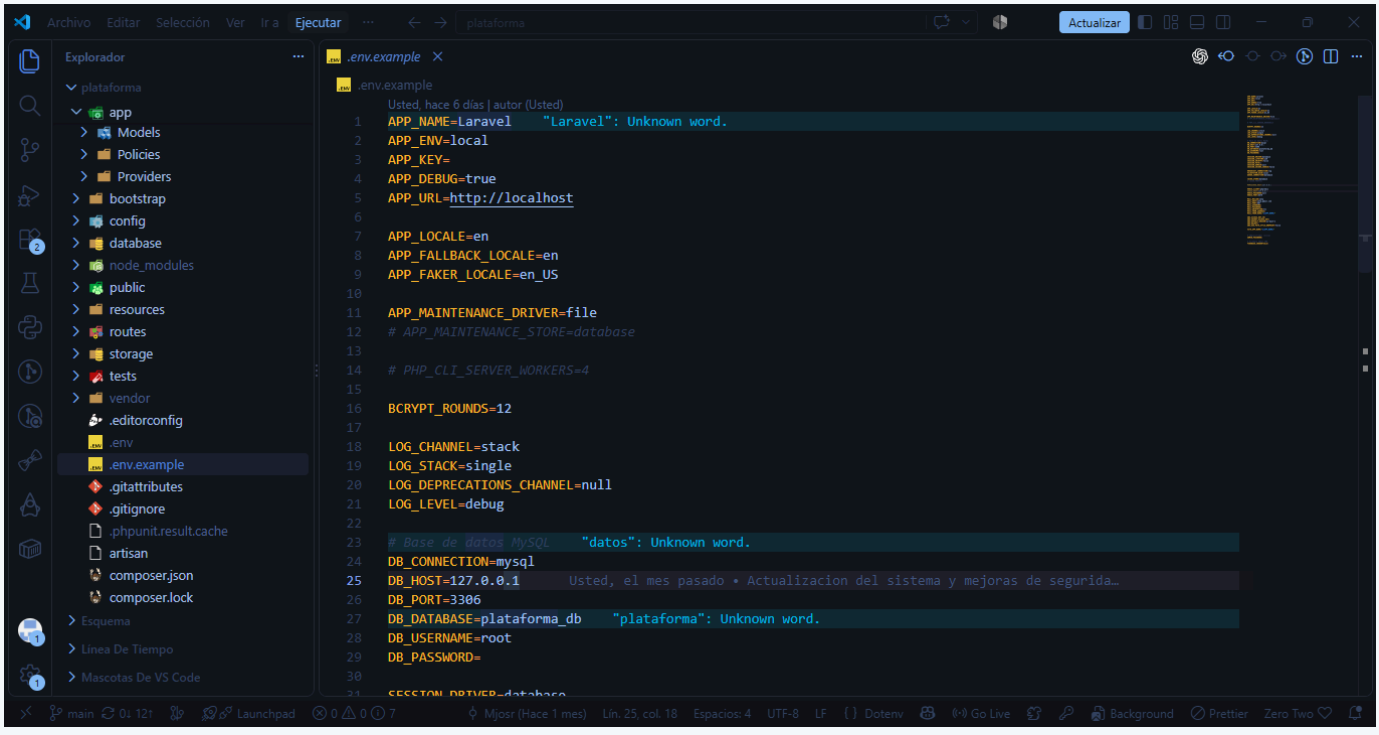
Variables principales

1. *APP_NAME*
2. *APP_ENV*
3. *APP_KEY*
4. *APP_DEBUG*
5. *APP_URL*
6. *DB_CONNECTION*
7. *DB_HOST*
8. *DB_PORT*
9. *DB_DATABASE*
10. *DB_USERNAME*
11. *DB_PASSWORD*

También se contemplan variables de servicios externos.

El archivo *.env* no debe subirse a un repositorio público. El archivo *.env.example* sirve como referencia sin valores sensibles.

.env.example



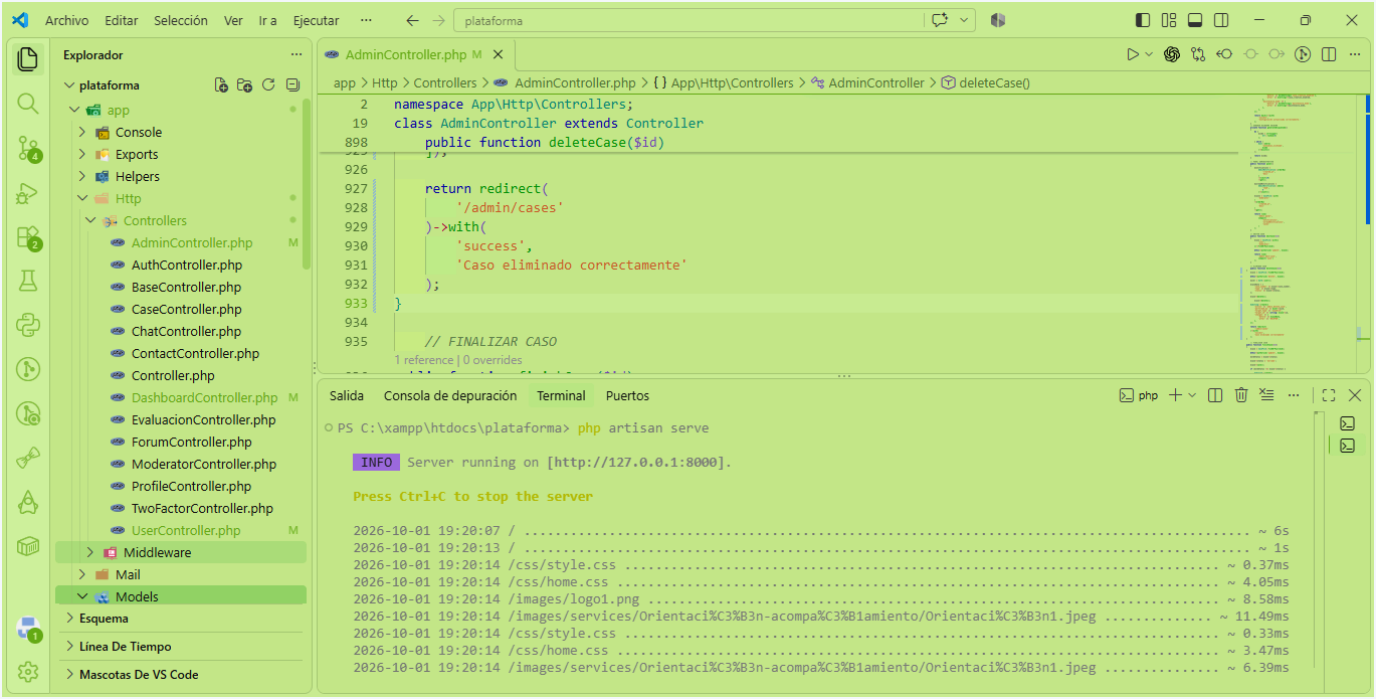
INSTALACIÓN DEL PROYECTO

La instalación de desarrollo contempla los siguientes pasos:

1. Obtener el proyecto: git clone URL_DEL_REPOSITORIO
2. Entrar al proyecto: cd plataforma
3. Instalar dependencias PHP: *composer install*

4. Instalar dependencias frontend: *npm install*
5. Configurar variables: *copy .env.example .env*
6. Generar clave: *php artisan key:generate*
7. Configurar base de datos en *.env*
8. Ejecutar migraciones: *php artisan migrate*
9. Compilar recursos: *npm run build*
10. Ejecutar la aplicación: *php artisan serve*

Terminal con php artisan serve



Plataforma funcionando en navegador



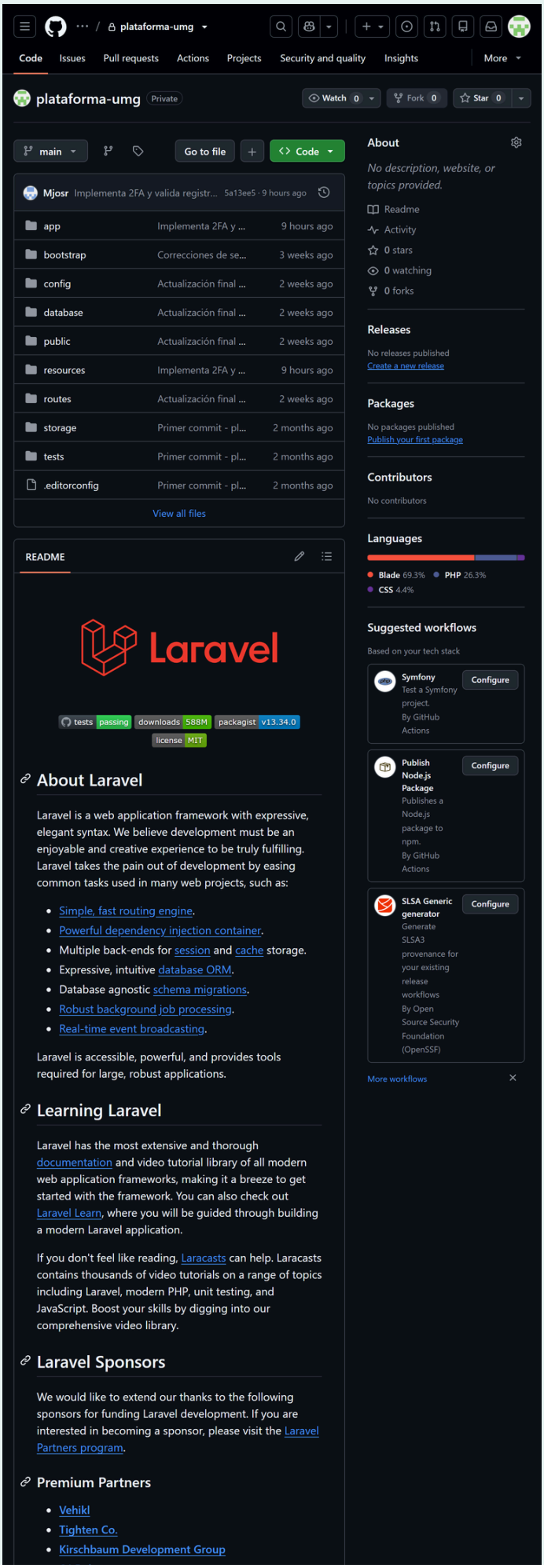
CONTROL DE VERSIONES

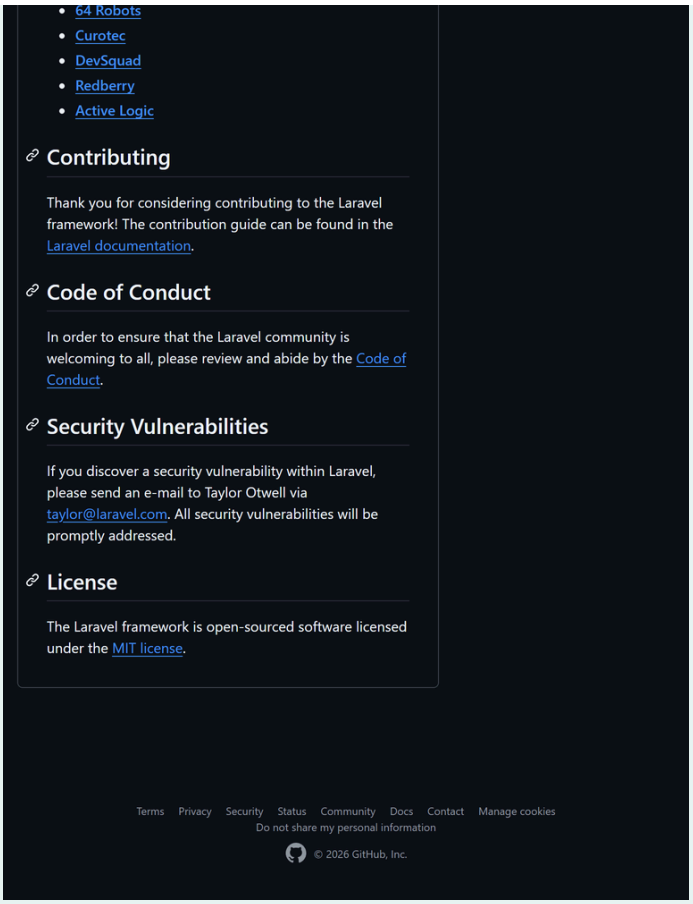
Git/*GitHub* permite registrar cambios y facilitar la continuidad del desarrollo de la plataforma.

Flujo de trabajo: Modificar código → *git status* → *git add .* → *git commit* → *git push* → *GitHub*

Comando	**Propósito**
git status	Consultar el estado de los cambios.
git add .	Preparar los cambios para el registro.
git commit -m "Descripción del cambio"	Registrar los cambios con una descripción.
git push	Enviar los cambios al repositorio de GitHub.

Repositorio de GitHub





DESPLIEGUE EN *DIGITALOCEAN*

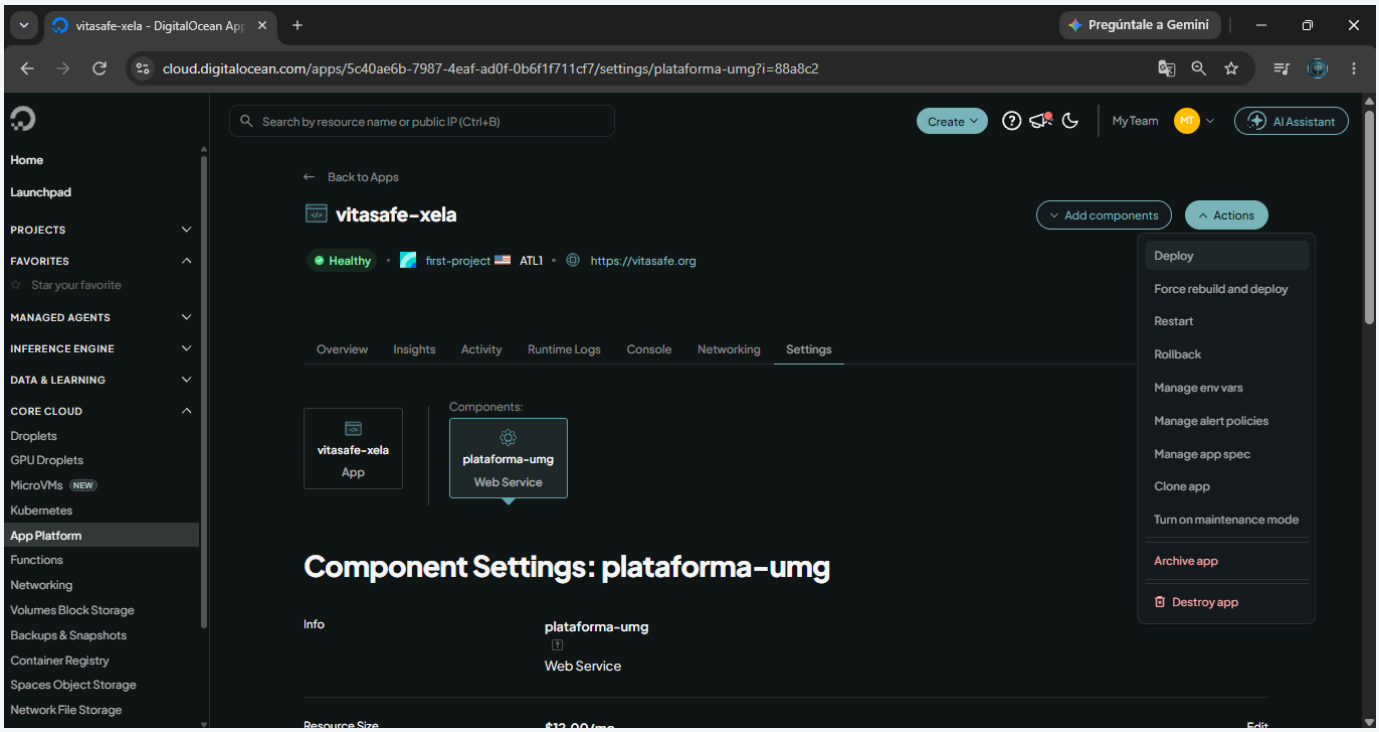
DigitalOcean se contempla como infraestructura de producción para ejecutar la aplicación sobre un servidor *Linux*.

Usuario → Navegador *web* → *HTTPS* → *DigitalOcean* → Servidor *Linux* → *Nginx* → *PHP* → *Laravel* → Aplicación → *MySQL*

Firebase Authentication y *Google Maps API* mantienen funciones independientes de la infraestructura principal de la aplicación.

La infraestructura definitiva puede variar según la configuración final.

DigitalOcean → Droplet → panel del servidor



SEGURIDAD

La seguridad de la plataforma contempla los siguientes elementos:

1. **Autenticación:** Control del acceso mediante *Firebase Authentication.*
2. **Control por roles:** Diferenciación de funciones entre *USER, MODERATOR* y *ADMIN.*
3. **Middleware de autorización:** Verificación de permisos antes del procesamiento de las rutas.
4. **Validación de solicitudes:** Revisión de la información recibida por la aplicación.
5. **Variables de entorno:** Separación de configuraciones y valores sensibles del código.
6. **Protección frente a intentos repetidos de inicio de sesión.** Después de cinco intentos fallidos, la cuenta puede quedar bloqueada por fuerza bruta. El sistema registra la causa del bloqueo. Cuando un administrador restaura una cuenta bloqueada por este motivo, se restablecen los contadores correspondientes y se exige al usuario cambiar su contraseña después de iniciar sesión.
7. **Auditoría:** Seguimiento de acciones mediante registros de auditoría.
8. **Protección de sesiones.** Las sesiones se almacenan mediante el controlador de base de datos de *Laravel* y se mantiene habilitado el cifrado de sesión. La plataforma incorpora un control de inactividad de quince minutos. En el caso del administrador, la verificación administrativa se invalida y debe renovarse antes de continuar con las funciones protegidas.
9. **Cifrado de información sensible.** Se utilizan atributos cifrados de Laravel para determinados campos que contienen información sensible: *case_files.description, case_comments.comment, chat_messages.message, mensajes_foro.content, evaluacion_respuestas.respuesta* y *crisis_reports.description.* Este mecanismo protege los valores almacenados y permite que la aplicación los descifre mediante el modelo correspondiente.
10. **Protección de solicitudes y contenido.** Se utilizan mecanismos de protección *CSRF* en formularios y solicitudes que lo requieren. Las vistas *Blade* emplean escape de salida en los lugares revisados, y la interfaz de chat utiliza *textContent* para presentar mensajes como texto. Las consultas de datos

revisadas utilizan *Eloquent* o consultas parametrizadas, sin evidencia de uso de consultas *SQL* sin procesar en la revisión efectuada.

11. **Cabeceras de seguridad.** Se verificó localmente la presencia de políticas para impedir la detección incorrecta de tipos de contenido, limitar la presentación en marcos, controlar el referente y restringir determinadas capacidades del navegador. También se comprobó la presencia de una política de seguridad de contenido (*CSP*).
12. **Limitaciones pendientes.** La política CSP mantiene las directivas '*unsafe-inline*' para *scripts* y estilos. Por ello, no debe afirmarse que se ha eliminado por completo la ejecución de código en línea. La política requiere una revisión adicional para endurecerla sin afectar las funcionalidades legítimas de la plataforma.
13. ***HTTPS* en producción:** Protección de la comunicación en el entorno productivo.
14. **Separación de credenciales:** Evitar la exposición de valores sensibles.
15. **Control administrativo:** Gestión de usuarios, configuración y actividades autorizadas.

RESPALDO Y MANTENIMIENTO

El mantenimiento debe considerar revisiones periódicas de Laravel, las dependencias *PHP* y *JavaScript*, la base de datos, el servidor, *HTTPS*, los servicios externos, la auditoría, el espacio del servidor y las copias de seguridad.

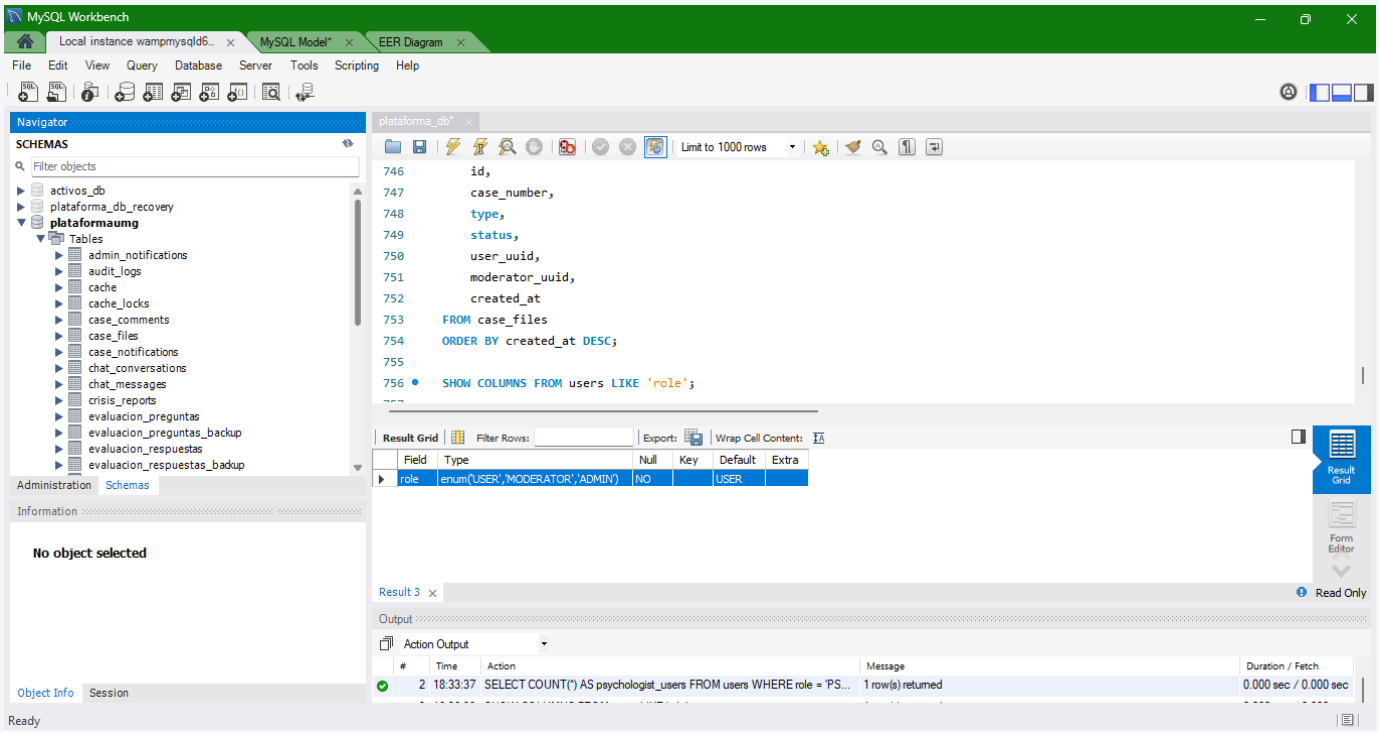
Aspectos de revisión

1. Revisar Laravel.
2. Revisar dependencias *PHP* y *JavaScript*.
3. Revisar la base de datos.
4. Revisar el servidor y el espacio disponible.
5. Revisar *HTTPS* y los servicios externos.
6. Revisar los registros de auditoría.
7. Verificar las copias de seguridad.

Antes de realizar cambios importantes se recomienda respaldar la base de datos.

Procedimiento: Realizar respaldo → Aplicar modificación → Probar aplicación → Verificar datos → Registrar cambio

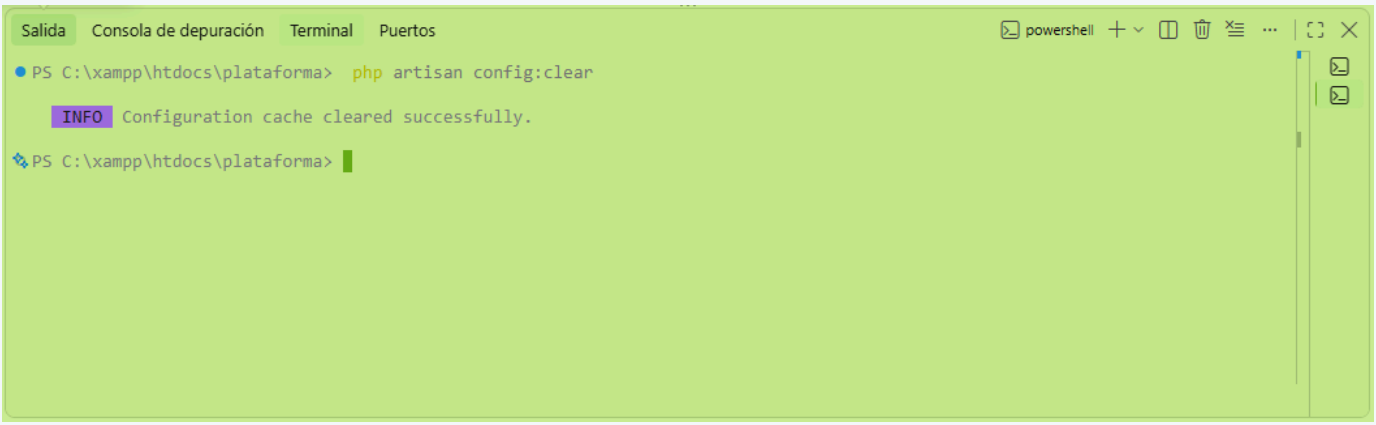
MySQL Workbench



SOLUCIÓN DE PROBLEMAS

Problema	Solución o revisión
La aplicación no inicia	Verificar *php artisan serve* y la versión de *PHP*.
Error de dependencias	Ejecutar *composer install* y revisar *composer.json.*
Error de base de datos	Revisar *MySQL*, la base, el usuario, la contraseña, el host y el puerto.
Error de configuración	Revisar *.env* y ejecutar *php artisan config:clear* y *php artisan cache:clear.*
Problemas con *Firebase*	Revisar el proyecto correcto, Authentication habilitado, proveedores, configuración y dominio autorizado.

Terminal con php artisan config:clear



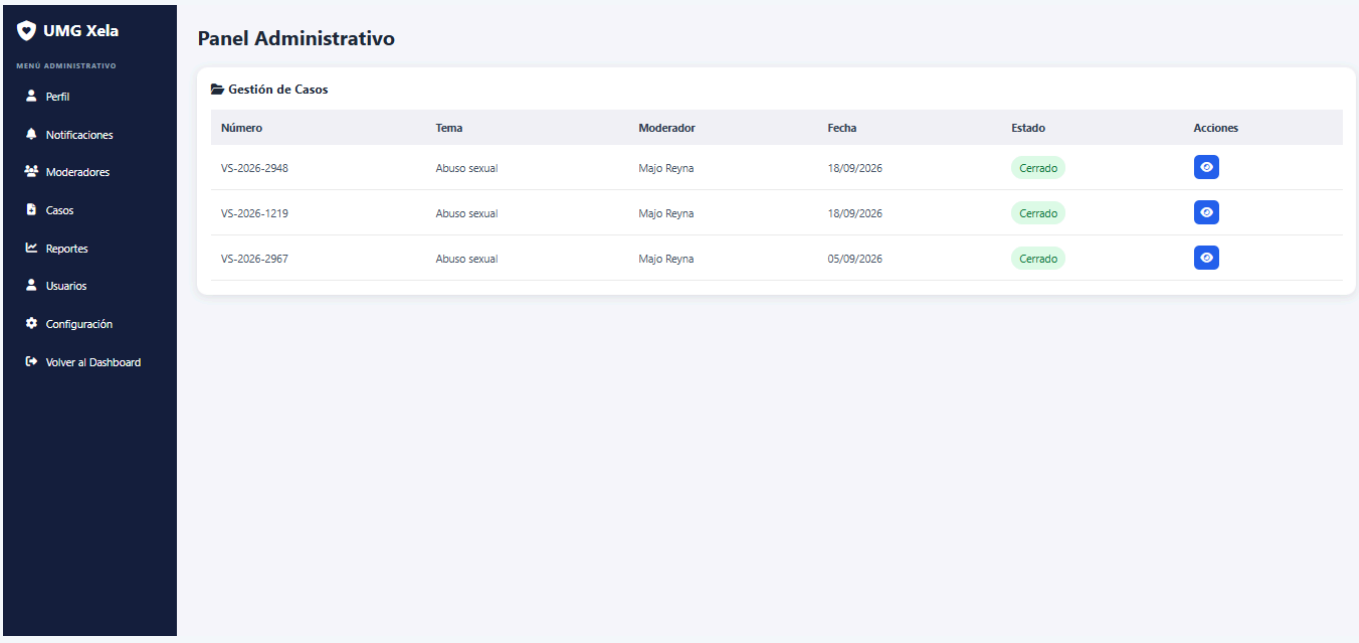
ESTRUCTURA DE ADMINISTRACIÓN

El módulo administrativo reúne funciones de gestión y consulta para la operación de la plataforma.

Funciones administrativas

1. Gestión de usuarios.
2. Gestión de moderadores.
3. Gestión de casos.
4. Gestión de configuración.
5. Gestión de notificaciones.
6. Consulta de auditoría.
7. Consulta y generación de informes.

Panel de Administrador



AUDITORÍA

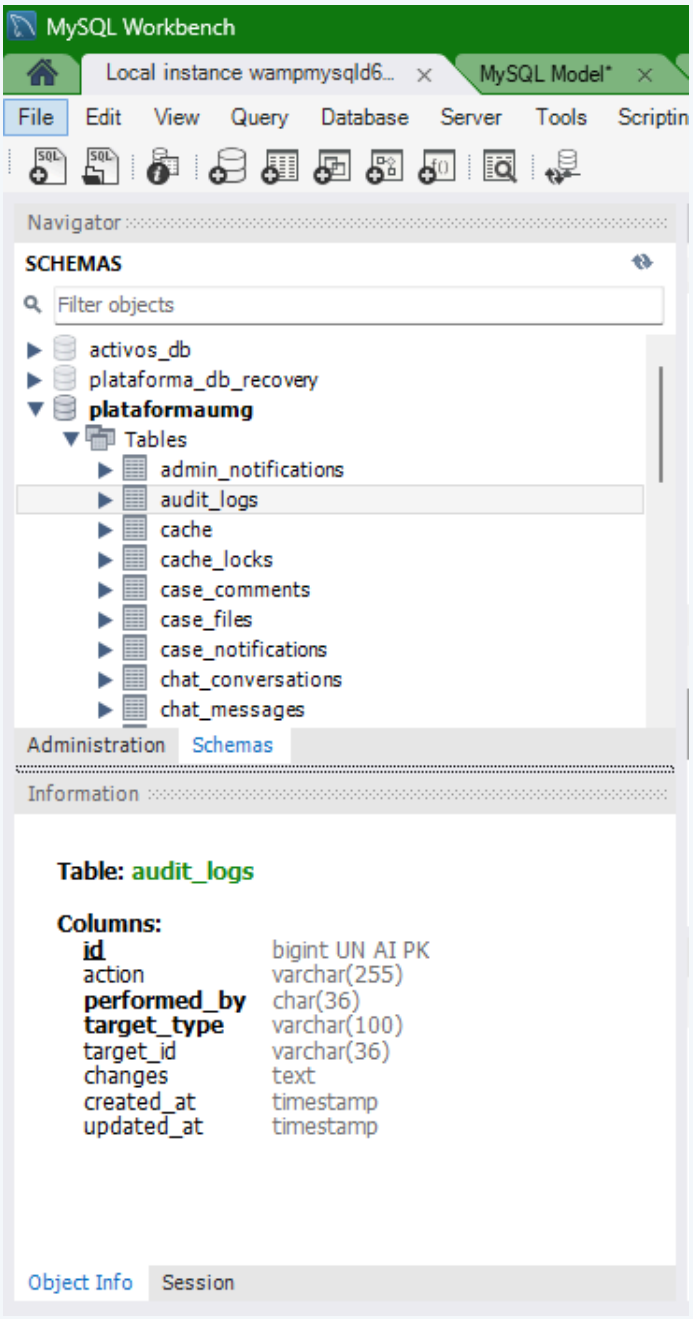
La plataforma dispone de la tabla *audit_logs* para registrar acciones administrativas y eventos de seguridad relevantes. En el código revisado se identificaron llamadas de registro para operaciones como la gestión de usuarios y moderadores, cambios de estado y eliminación o cierre de casos, comentarios de casos, modificaciones de configuración, gestión de evaluaciones, restricciones del foro, cambios de perfil y contraseña, recuperación de contraseñas y administración de la autenticación de dos factores.

Esto respalda que el flujo de la aplicación añade registros sin modificarlos mediante esos métodos; no demuestra que los registros sean inmutables frente a usuarios con acceso directo a la base de datos.

Los mensajes de chat, las publicaciones habituales del foro y las respuestas e intentos de evaluación se almacenan en sus respectivas tablas funcionales. No todas estas operaciones generan un registro independiente en *audit_logs*. Por ello, debe distinguirse entre los datos operativos de la plataforma y el registro de auditoría de acciones administrativas y de seguridad.

Los registros de auditoría facilitan la trazabilidad y la revisión de actividades relevantes. La protección frente a modificaciones directas en la base de datos requiere controles adicionales de permisos, acceso administrativo, respaldos y, si se necesita inmutabilidad, mecanismos específicos a nivel de infraestructura.

MySQL Workbench → *audit_logs*



INFORMES

Los informes organizan información registrada en la plataforma para su consulta o presentación cuando corresponda.

Flujo de informes: Registro → Almacenamiento → Consulta → Filtrado → Procesamiento → Generación → Exportación

El sistema puede generar información para consulta o presentación cuando corresponda. El proceso parte de los registros almacenados y continúa con la consulta, el filtrado, el procesamiento, la generación y la exportación.

Panel de reportes

← Volver al panel

Reportes Administrativos

Consulta las estadísticas y genera reportes de los casos registrados en UMG.

Filtrar por fecha

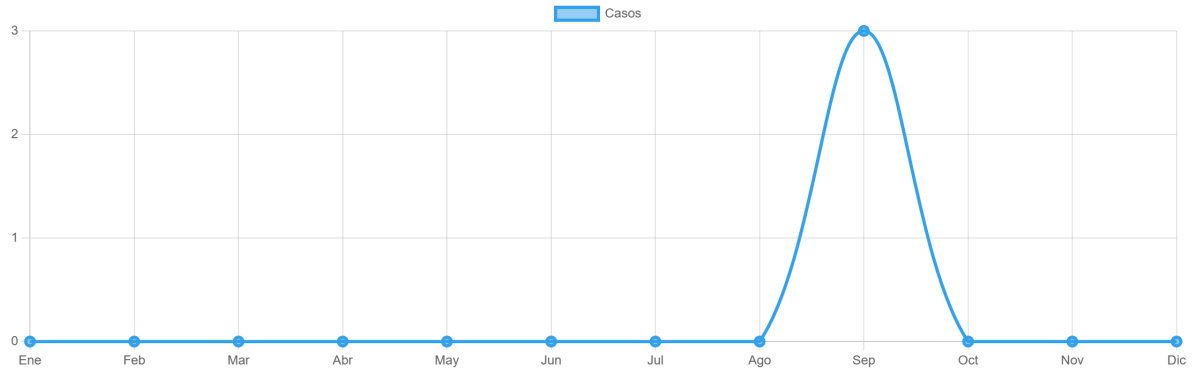
Fecha inicial: dd/mm/aaaa

Fecha final: dd/mm/aaaa

Filtrar

Casos abiertos	Casos cerrados	Usuarios registrados	Moderadores activos
1	2	0	1

Casos registrados por mes

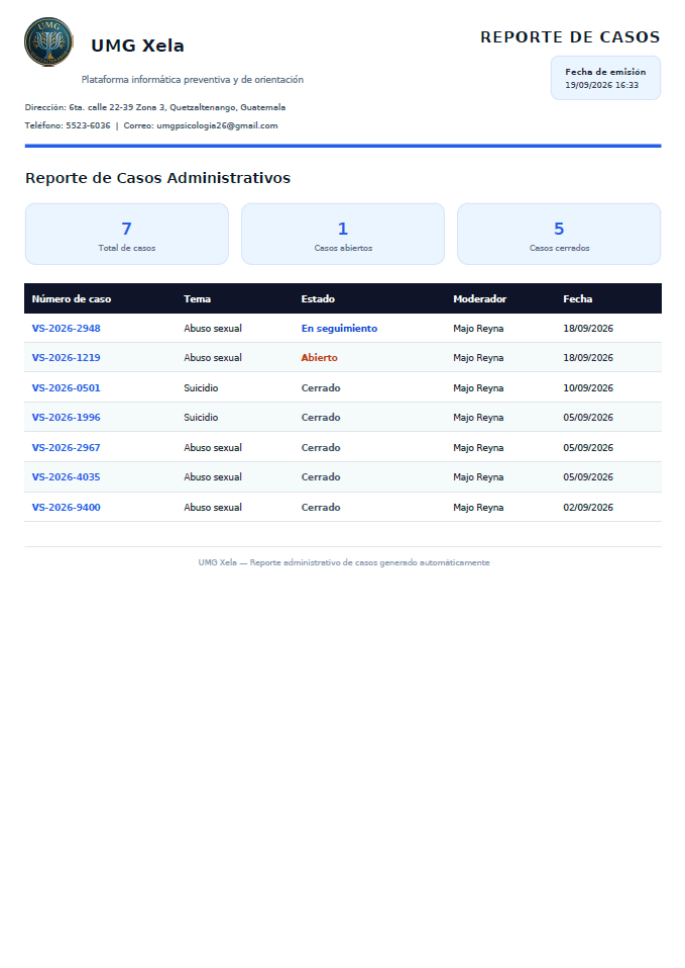


Exportar reporte

Exportar PDF

Exportar Excel

PDF generado



UMG Xela

Plataforma informática preventiva y de orientación

Dirección: 6ta. calle 22-39 Zona 3, Quetzaltenango, Guatemala

Teléfono: 5523-6036 | Correo: umgpsicologia26@gmail.com

REPORTE DE CASOS

Fecha de emisión
19/09/2026 16:33

Reporte de Casos Administrativos

7	1	5
Total de casos	Casos abiertos	Casos cerrados

Número de caso	Tema	Estado	Moderador	Fecha
VS-2026-2948	Abuso sexual	En seguimiento	Majo Reyna	18/09/2026
VS-2026-1219	Abuso sexual	Abierto	Majo Reyna	18/09/2026
VS-2026-0501	Suicidio	Cerrado	Majo Reyna	10/09/2026
VS-2026-1996	Suicidio	Cerrado	Majo Reyna	05/09/2026
VS-2026-2967	Abuso sexual	Cerrado	Majo Reyna	05/09/2026
VS-2026-4035	Abuso sexual	Cerrado	Majo Reyna	05/09/2026
VS-2026-9400	Abuso sexual	Cerrado	Majo Reyna	02/09/2026

UMG Xela — Reporte administrativo de casos generado automáticamente

ARQUITECTURA DE PRODUCCIÓN

La arquitectura propuesta representa la comunicación entre el usuario final, la aplicación, la base de datos y los servicios externos.

Internet → Usuario final → Navegador *web* → *HTTPS* → *DigitalOcean* → *Nginx* → *PHP* → *Laravel* → *MySQL*

Firebase Auth y *Google Maps API* se consideran servicios externos de la arquitectura de producción.

COMPONENTES Y SERVICIOS

Componente o servicio	Función
Laravel	*Framework* principal.
PHP	Lenguaje de programación.
Blade	Construcción de vistas.
Eloquent ORM	Acceso a datos.
MySQL	Persistencia.
Firebase Authentication	Autenticación.
Google Maps API	Geolocalización.
Git/*GitHub*	Control de versiones.
DigitalOcean	Infraestructura de producción.
Nginx	Servidor *web* previsto.
Composer	Dependencias *PHP*.
NPM	Dependencias *frontend*.

RECOMENDACIONES TÉCNICAS

1. Mantener actualizado *Laravel* y sus dependencias compatibles.
2. Mantener *PHP* soportado.
3. Realizar respaldos periódicos.
4. No almacenar credenciales en el código.
5. Proteger el archivo *.env*.
6. Revisar los registros de auditoría.
7. Actualizar *Git*.
8. Documentar los cambios.
9. Verificar los servicios externos.
10. Probar los cambios antes de producción.
11. Mantener *HTTPS*.
12. Revisar *DigitalOcean*.

CIERRE DEL MANUAL

La plataforma integra funcionalidades de autenticación, gestión de usuarios, administración de casos, mensajería, evaluaciones y configuración del sistema, implementadas sobre una arquitectura basada en Laravel y una base de datos *MySQL*.

La revisión de seguridad permitió documentar controles de autorización por roles, autenticación de dos factores para *ADMIN* y *MODERATOR*, bloqueo por intentos fallidos de inicio de sesión, recuperación y cambio de contraseña, protección de sesiones, cifrado de campos sensibles, controles *CSRF*, cabeceras de seguridad y registro de acciones relevantes.

La auditoría también identificó aspectos que requieren atención antes de considerar completado el endurecimiento de seguridad, entre ellos la revisión de las directivas en línea de la política *CSP* y la validación de la configuración definitiva de producción.

Este manual debe mantenerse sincronizado con las migraciones, los modelos, los controladores, las rutas y la configuración de la versión liberada. Cualquier modificación futura que afecte a la autenticación, la estructura de la base de datos, los permisos o el tratamiento de información sensible deberá reflejarse en la documentación técnica correspondiente.